QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Week 5 – Sem 2 – 2026

Prof Niko Suenderhauf

Welcome to Week 5!

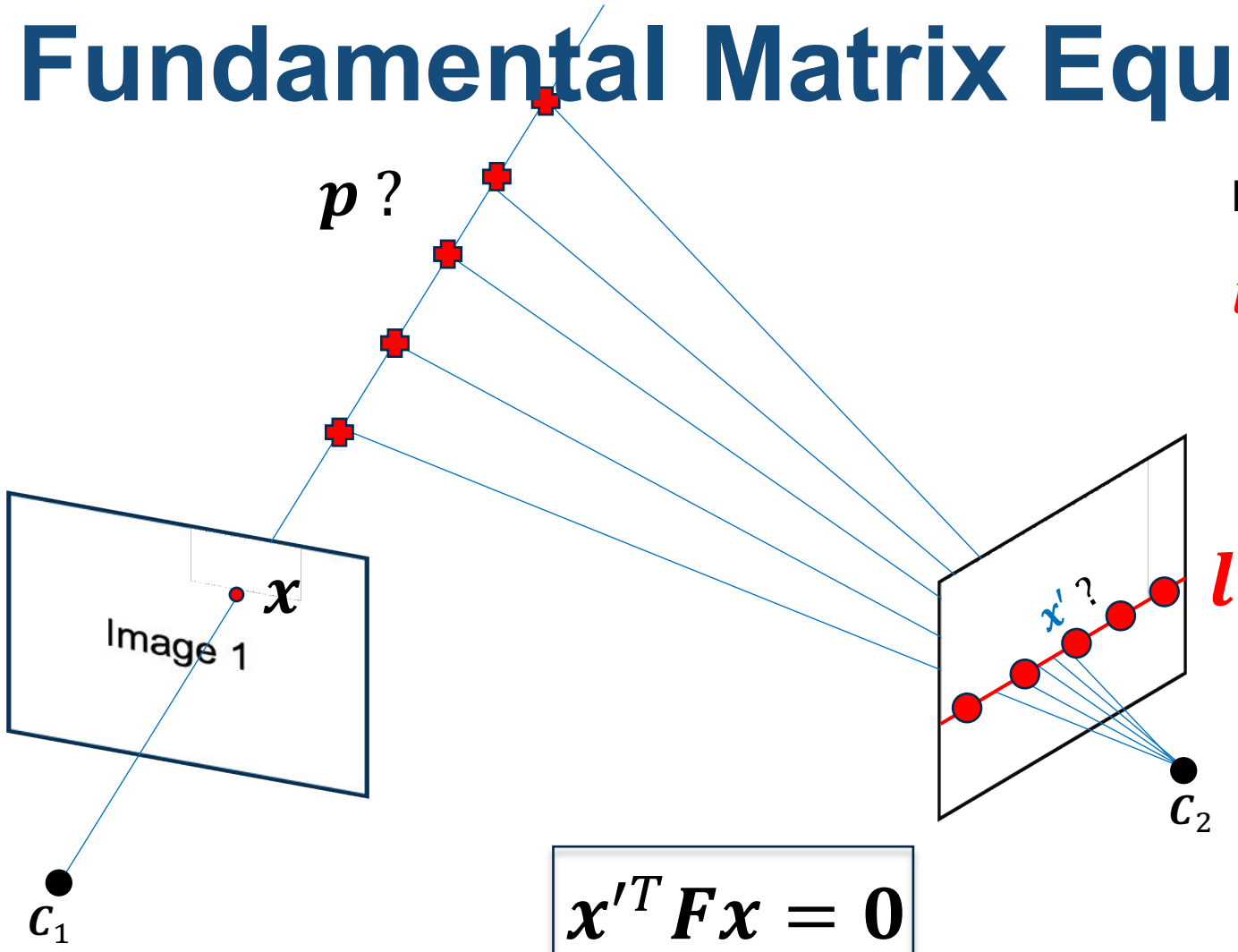


CRICOS No.00213J



Recap

The Fundamental Matrix Equation



F is the **Fundamental Matrix** (3x3)

l is called the **Epipolar Line** of x.

$$l = Fx$$

x' must be on l!

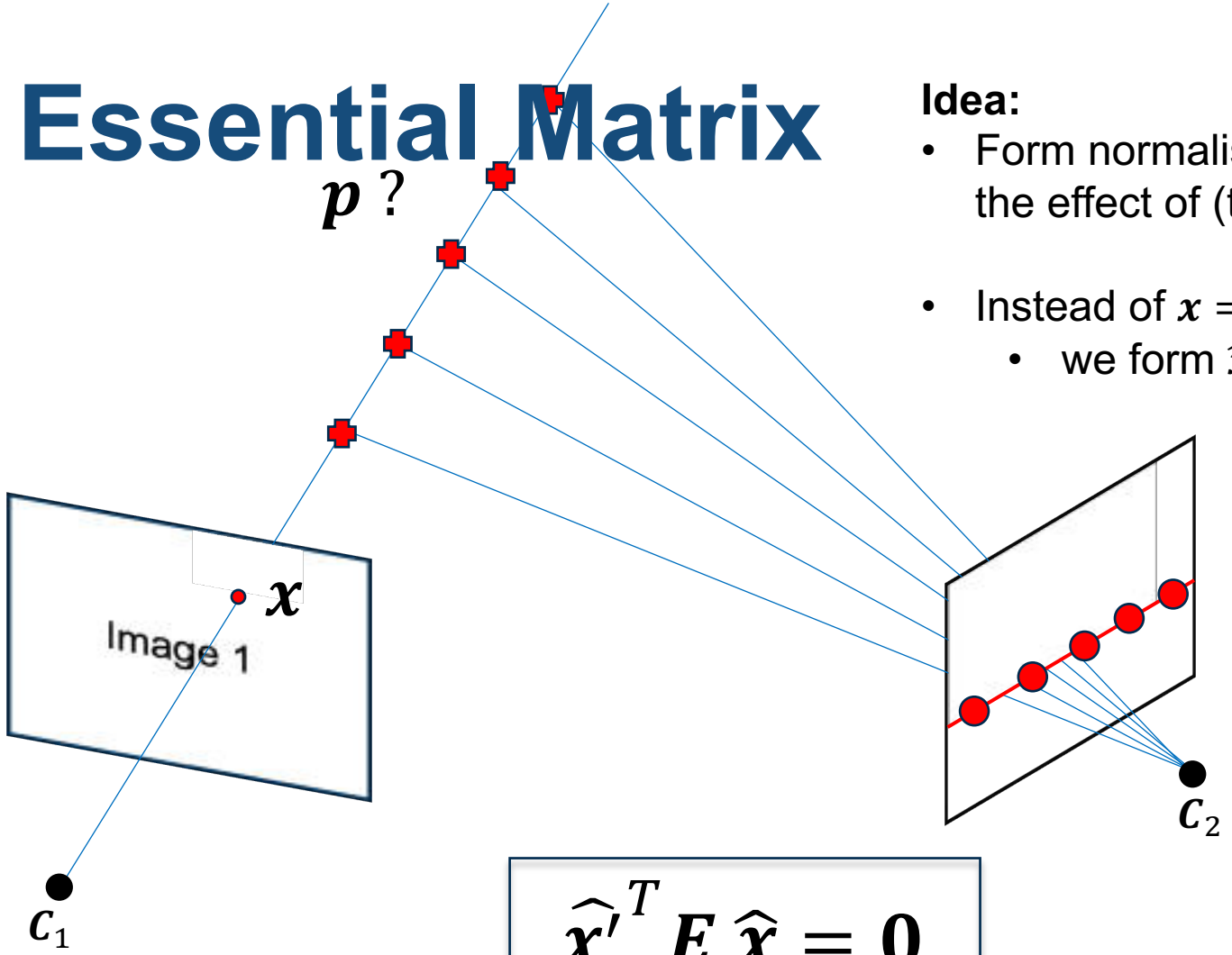
$$x'l = 0$$

Remember from the discussion of homogeneous coordinates!

$$x'^T Fx = 0$$

This is true for **any x** and corresponding **x'**!

The Essential Matrix



Idea:

- Form normalised image coordinates, by removing the effect of (the known) K.

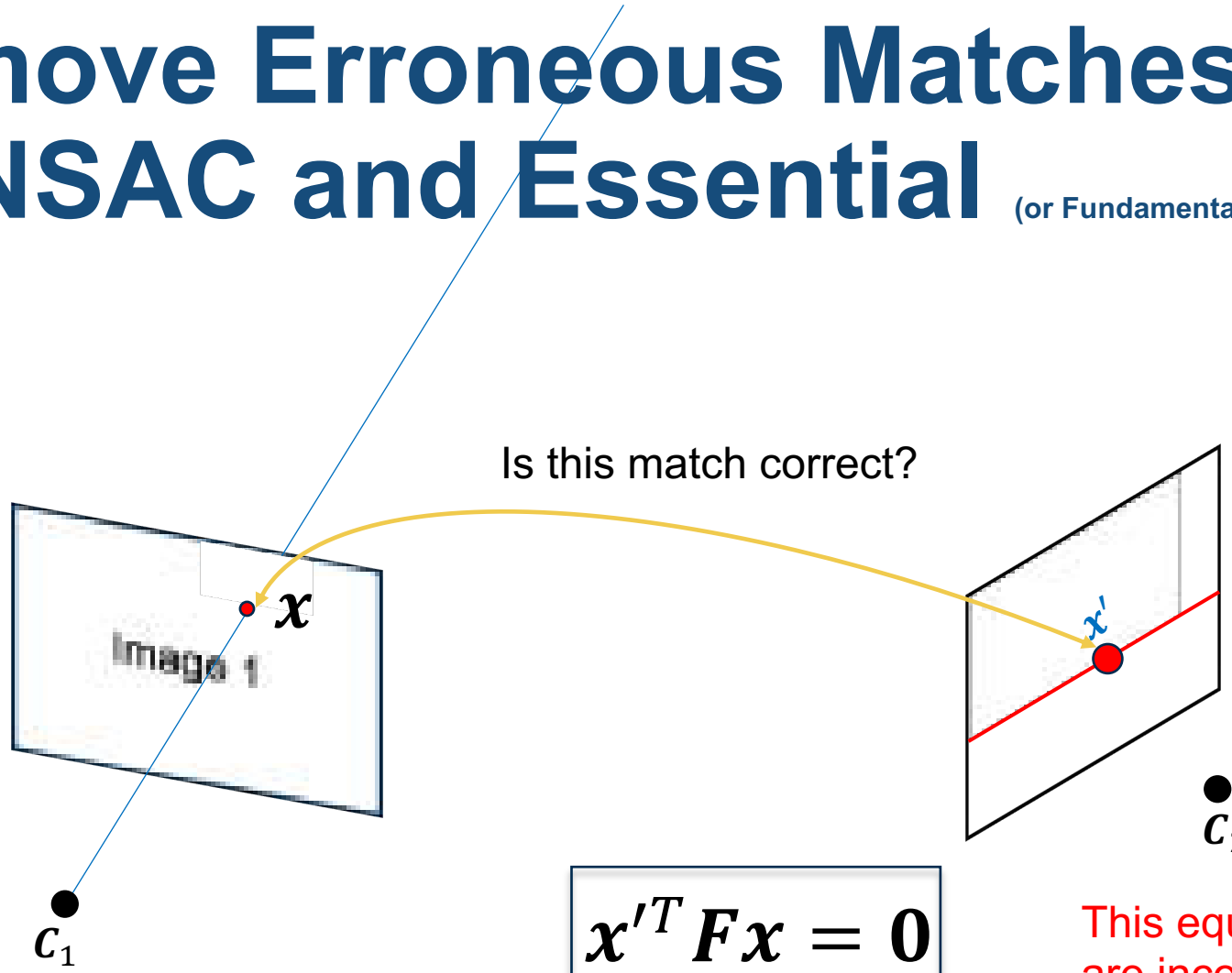
- Instead of $\boldsymbol{x} = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p}$ …
  - we form $\hat{\boldsymbol{x}} = \boldsymbol{K}^{-1}\boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p} = [\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p}$

$$\hat{\boldsymbol{x}}'^{T}\,\boldsymbol{E}\,\hat{\boldsymbol{x}} = \boldsymbol{0}$$

This is true for **any x** and corresponding **x'**!

CRICOS No.00213J

Remove Erroneous Matches with RANSAC and Essential (or Fundamental) Matrix



Idea:

- Calculate F or E from a small subset of proposed matches
- Check which point pairs do not pass the test (equation does not hold)
- Repeat until sufficiently large subset passes the test

= Random Sample Consensus (RANSAC), 1981

$$x'^T F x = 0$$

This is true for **any x** and corresponding **x'**!

This equation will **not** hold if x and x' are incorrect matches!

Random Sample Consensus (RANSAC, 1981)

RANSAC:

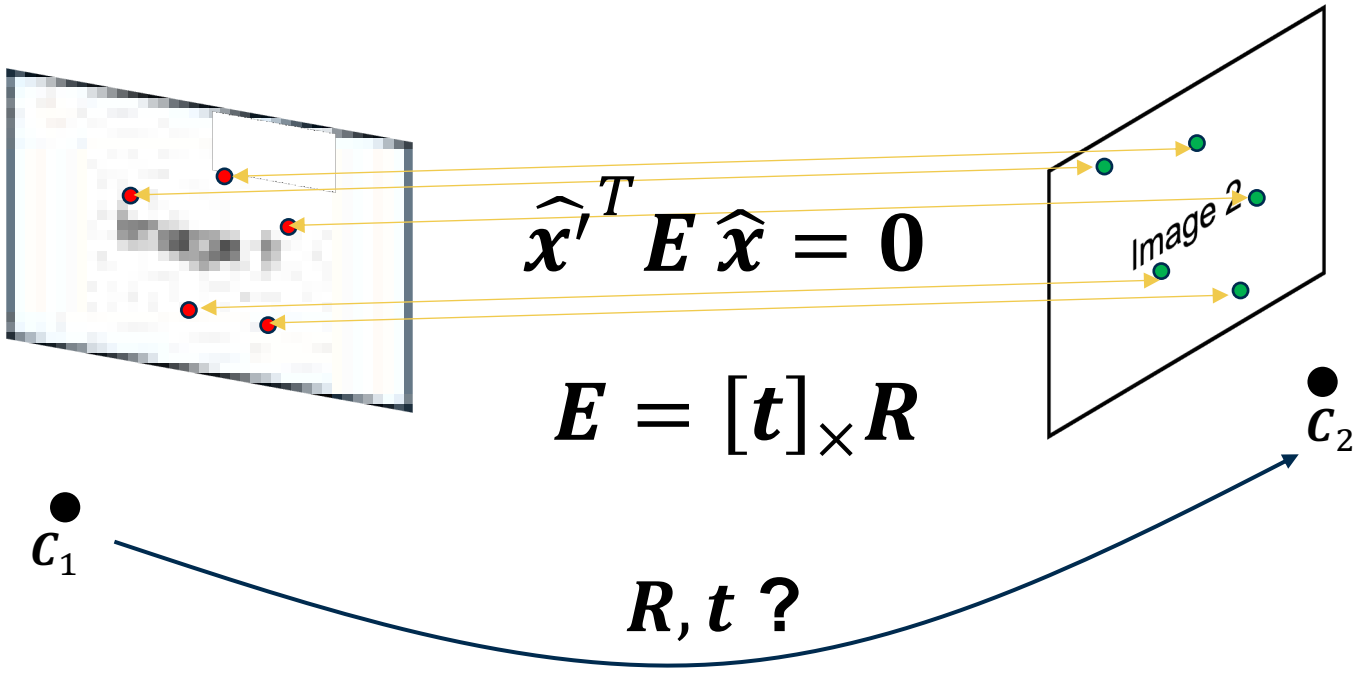
General setup: We want to estimate a model based on potentially noisy / erroneous data

1. *Calculate the model from a randomly selected minimal set (using as little of the data as possible)*
2. *Calculate the error between the model and the complete set of data*
3. *Data points with an error < threshold are "inliers" all data points with error > threshold are "outliers"*
4. *Repeat from 1 for k repetitions.*
5. *Best model is the one with the biggest inlier set.*

$$(\text{data} \rightarrow \text{sample} \rightarrow \text{fit model} \rightarrow \text{score} \rightarrow \text{repeat} \rightarrow \text{terminate})$$

CRICOS No.00213J

Summary: Motion from 2D-2D Matches

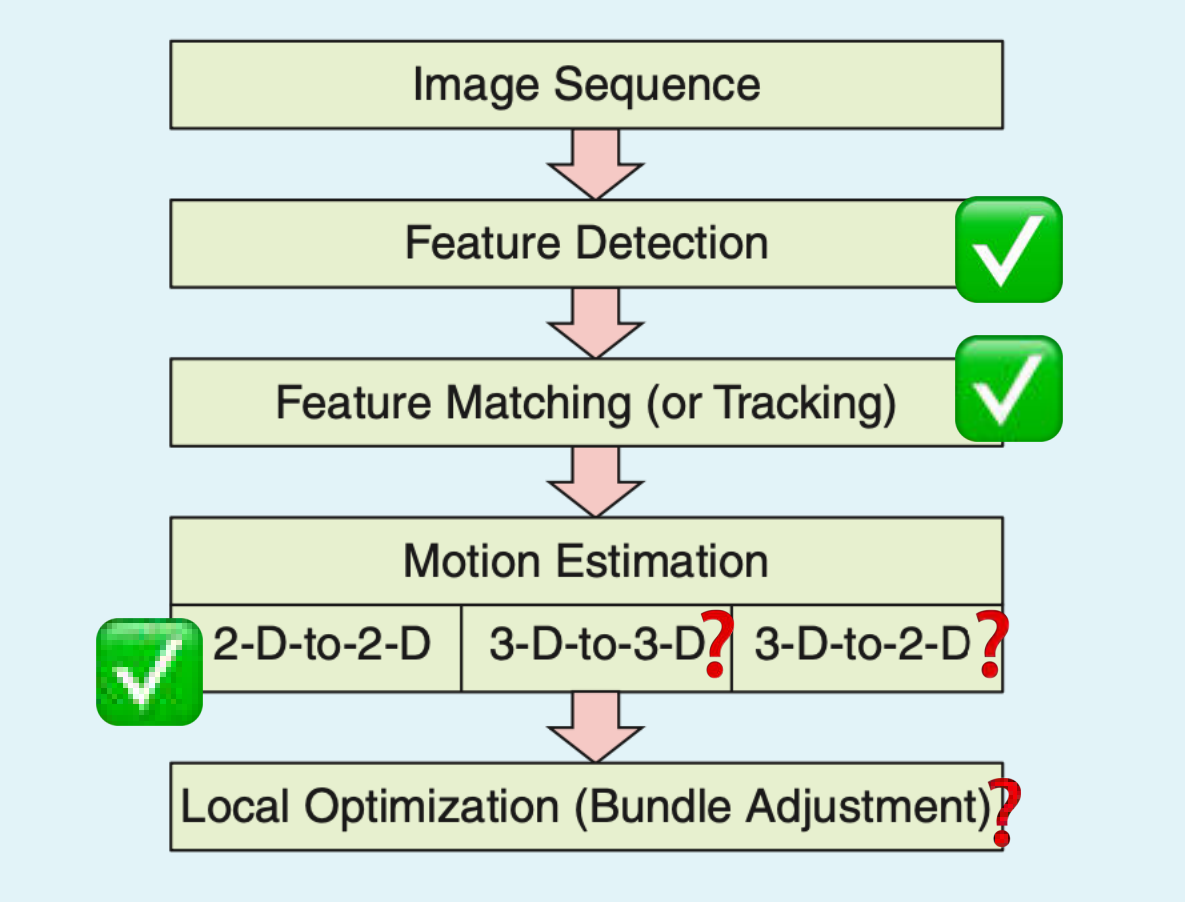


- Find interest points in both images.
- Match them.
- Calculate Essential Matrix using `findEssentialMat()`
- Find **R** and **t** using `recoverPose(E, points1, points2)`

- **R** is correct, but **t** is only the direction of the translation!
- The translation distance cannot be recovered without additional information.

CRICOS No.00213J

Recap – What we learned so far

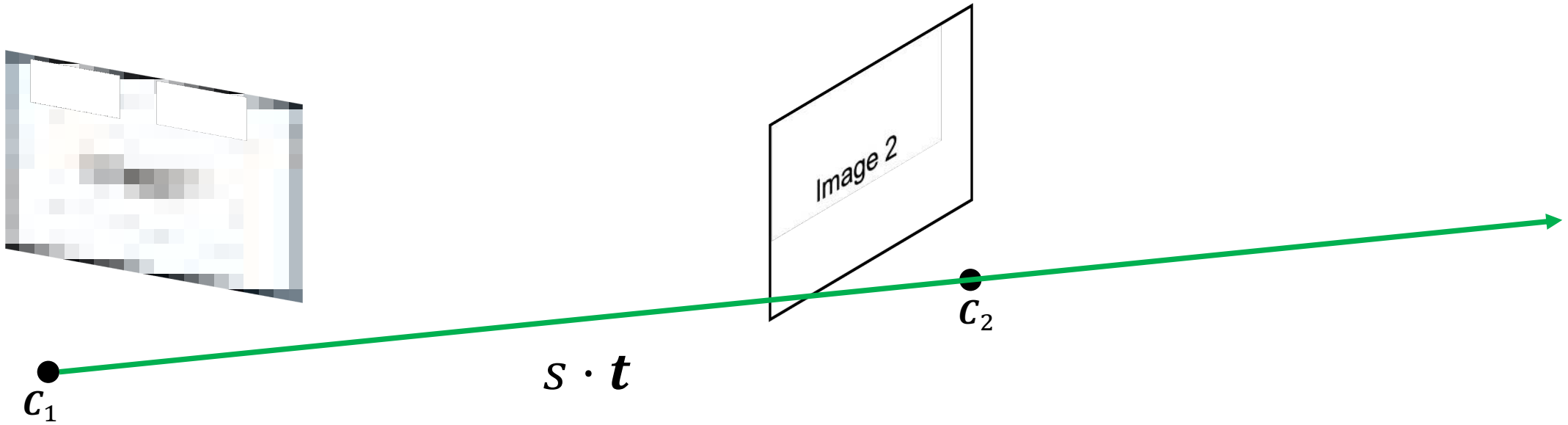




Camera Motion from 2D-2D Correspondences

An Important Limitation

- **t** can only be recovered up-to-scale! Direction yes, but not distance
- **R** will be correct



We know the second camera centre C_2 is somewhere on the vector $s \cdot t$, but we do not know where.
The unknown scale factor s cannot be recovered without additional information.

CRICOS No.00213J

Why is the scale ambiguous?

- From the Essential Matrix we can recover $\mathbf{R}$ and $\hat{\mathbf{t}}$
  - $\hat{\mathbf{t}}$ is a unit-length translation direction
  - The actual translation $\mathbf{t} = s \cdot \hat{\mathbf{t}}$ is unknown
    - we don't know the scale factor $s > 0$

- **Why?**
  - Scaling both the translation and the scene by the same factor leaves the image observations unchanged.

$$\boldsymbol{t}' = s \cdot \boldsymbol{t} \qquad \text{and} \qquad \boldsymbol{X}'_i = s \cdot \boldsymbol{X}_i$$

  - There is not enough information to uniquely determine the scale factor!

CRICOS No.00213J

Why is the scale ambiguous?

Project a world point into the 2nd camera

$$\lambda \cdot \boldsymbol{x}_i = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}]\boldsymbol{X}_i$$

$$\boldsymbol{K}(\boldsymbol{R}\boldsymbol{X}_i + \boldsymbol{t}) = \tilde{\boldsymbol{x}}_i = \lambda \cdot \begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix}$$

Scaling both the translation and the scene by the same factor leaves the image observations unchanged.

CRICOS No.00213J

Why is the scale ambiguous?

Project a world point into the 2nd camera

$$\lambda \cdot \boldsymbol{x}_i = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}]\boldsymbol{X}_i$$

$$\boldsymbol{K}(\boldsymbol{R}\boldsymbol{X}_i + \boldsymbol{t}) = \tilde{\boldsymbol{x}}_i = \lambda \cdot \begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix}$$

Let's scale the translation and the world:

$$\boldsymbol{t}' = s \cdot \boldsymbol{t} \text{ and } \boldsymbol{X}'_i = s \cdot \boldsymbol{X}_i$$

$$\lambda \cdot \boldsymbol{x}'_i = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}']\boldsymbol{X}'_i$$

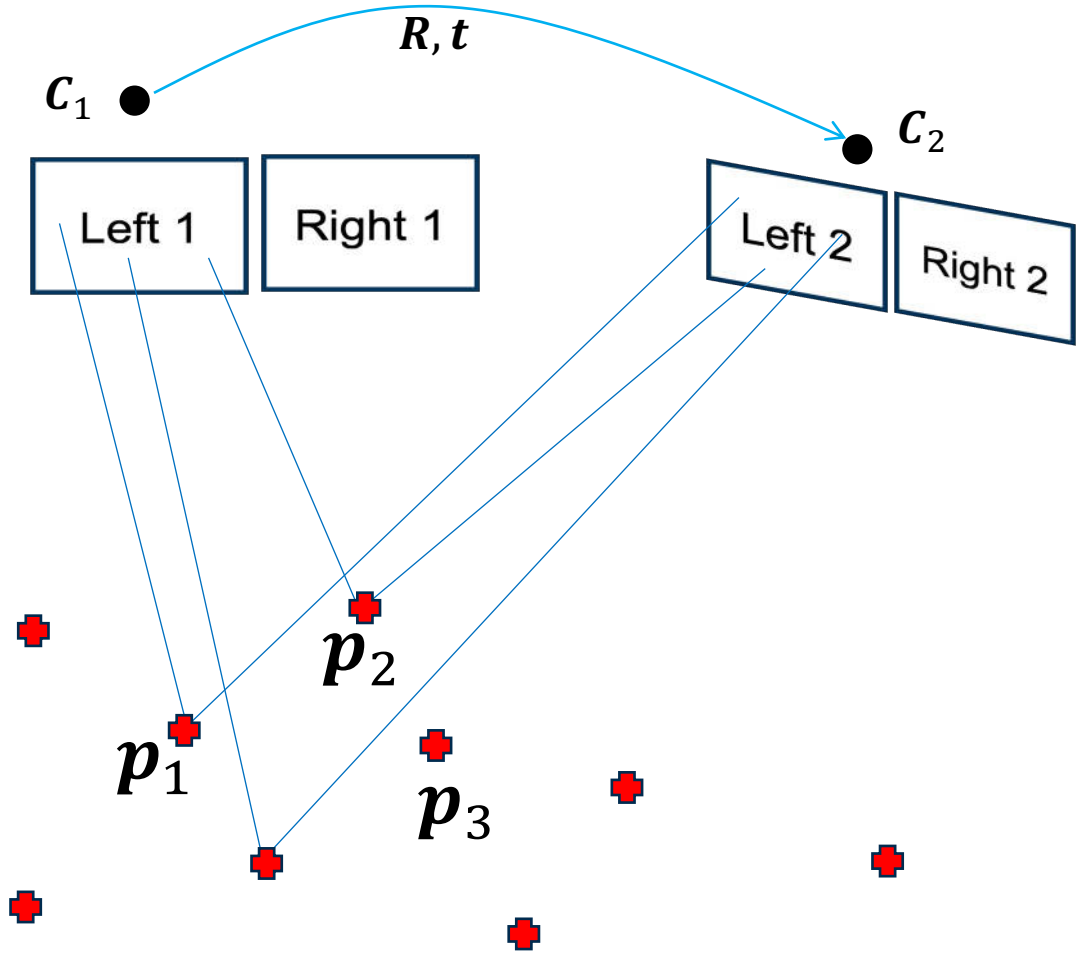
$$\boldsymbol{K}(\boldsymbol{R}s\boldsymbol{X}_i + s\boldsymbol{t}) = s\boldsymbol{K}(\boldsymbol{R}\boldsymbol{X}_i + \boldsymbol{t})$$

$$s\boldsymbol{K}(\boldsymbol{R}\boldsymbol{X}_i + \boldsymbol{t}) = \tilde{\boldsymbol{x}}'_i = s\lambda \cdot \begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix}$$

Scaling both the translation and the scene by the same factor leaves the image observations unchanged.

CRICOS No.00213J

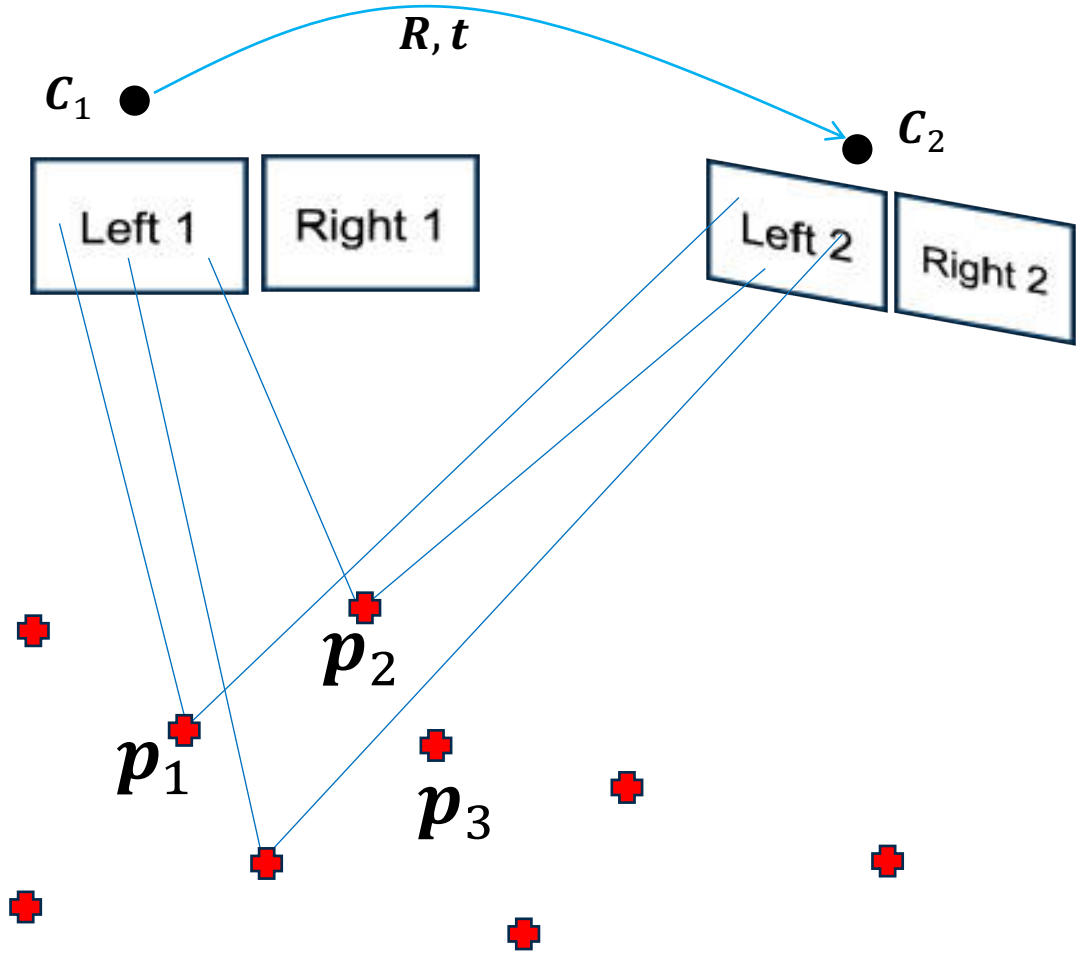
Recover Scale from Known 3D Points



- If we have access to the true depth of some points, we can recover the missing scale factor
  - E.g. with a stereo camera or depth camera

CRICOS No.00213J

Recover Scale from Known 3D Points

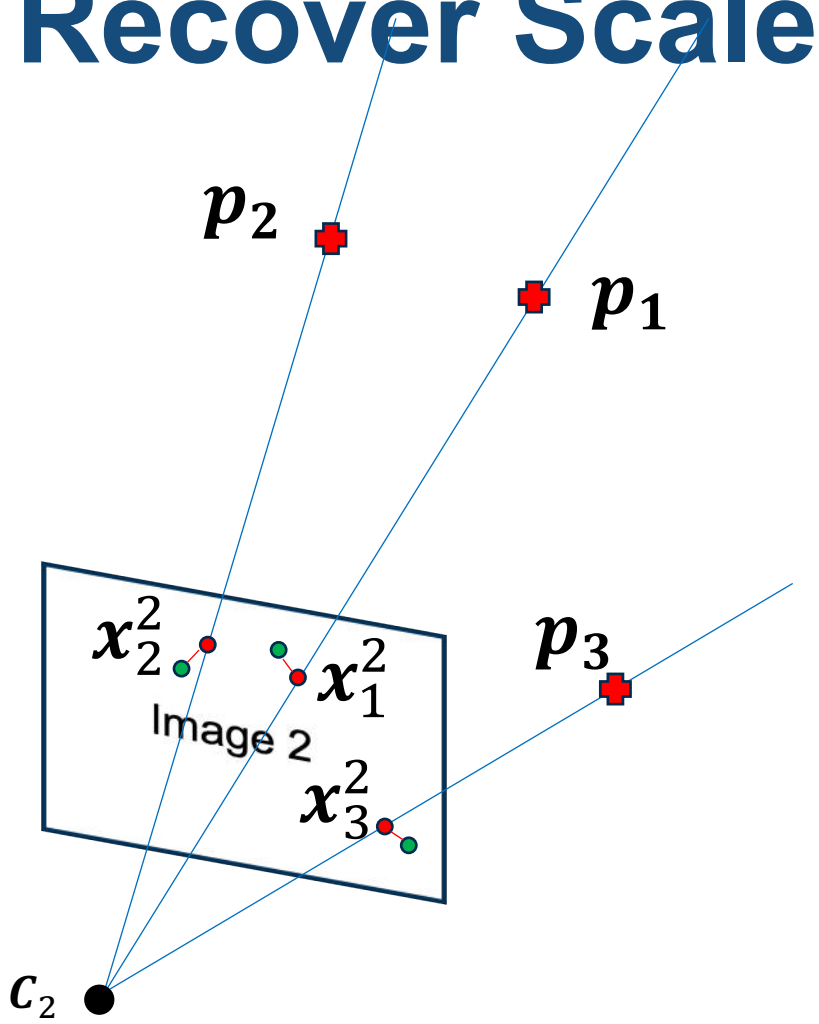


- Idea: We know the 3D coordinates p_i for some image points $\mathbf{x}_i^k$.
- We also know where that 3D point should project into the second camera, if we had the correct scale s:

$$\mathbf{x}_i^{k+1} = \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$$

CRICOS No.00213J

Recover Scale from Known 3D Points



- Idea: We know the 3D coordinates p_i for some image points $\mathbf{x}_i^k$.
- We also know where that 3D point should project into the second camera, if we had the correct scale s:

$$K(R \cdot p_i + s \cdot t)$$

- But if we have the wrong scale, we will observe the point at a different pixel.
- The difference is called the **reprojection error**
- Solution: Find the scale that minimises the reprojection error:

$$s* = \text{argmin}_s \sum_i \left\| x_i^{k+1} - K(R \cdot p_i + s \cdot t) \right\|^2$$

CRICOS No.00213J

Recover Scale from Known 3D Points

This is a **nonlinear least-squares optimisation** problem.

Problems of this structure are very common in robotic vision / computer vision.

See Practical on Friday on how to solve this using Scipy.

$$s* = \text{arg}min_s \sum_i \left\| \boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t}) \right\|^2$$

CRICOS No.00213J

Optimisation Problems – Template

- **Parameters:** What do we not know?
- **Prediction:** Given a guess, what measurement would we expect?
- **Residual/Error:** How different is the prediction from the observation?
- **Objective:** Which parameter values minimise the combined residuals?

Component	Example Scale recovery
Parameters	s
Prediction	projected pixel $\boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$
Residual / Error	Reprojection Error $\boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$
Objective	sum of squared reprojection errors $\sum_i \| \cdots \|^2$

$$s* = \text{arg}min_s \sum_i \left\| \boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t}) \right\|^2$$

CRICOS No.00213J

Different Problems, Different Solution

Problem structure	What we do	Typical tool
Exact, directly invertible equations	Solve for the unknowns	Rearrangement or linear solve
Noisy linear equations	Minimise linear residuals	QR-decomposition or SVD
Noisy nonlinear equations	Iteratively minimise residuals	Gauss–Newton or Levenberg-Marquardt
Outliers are present	Limit or reject their influence	Robust loss function or RANSAC

1. Is the measurement model exact or approximate?
2. Is it linear or nonlinear?
3. Are outliers expected?

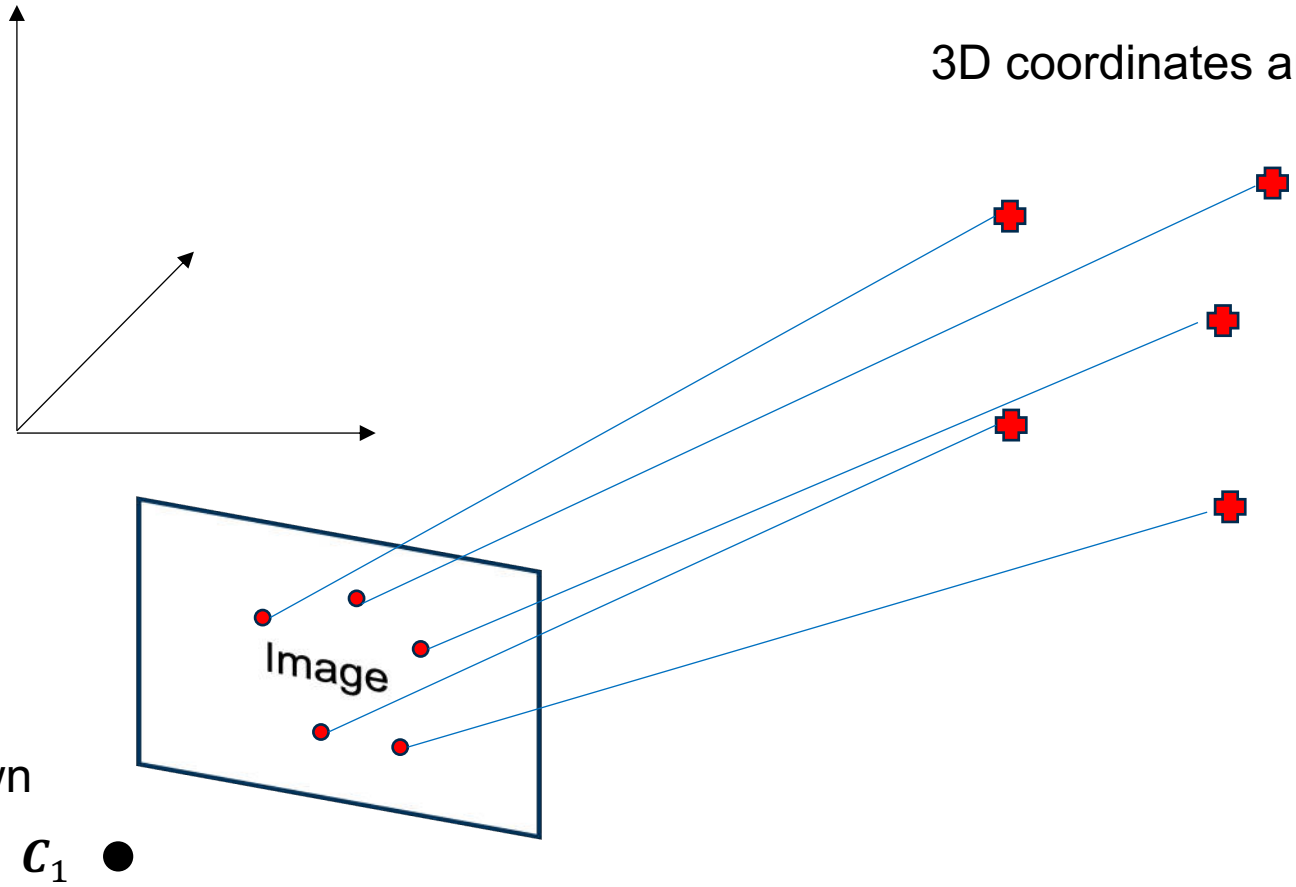
CRICOS No.00213J



Camera motion from 3D-2D Correspondences

Localisation from 3D-2D Correspondences

3D coordinates are known

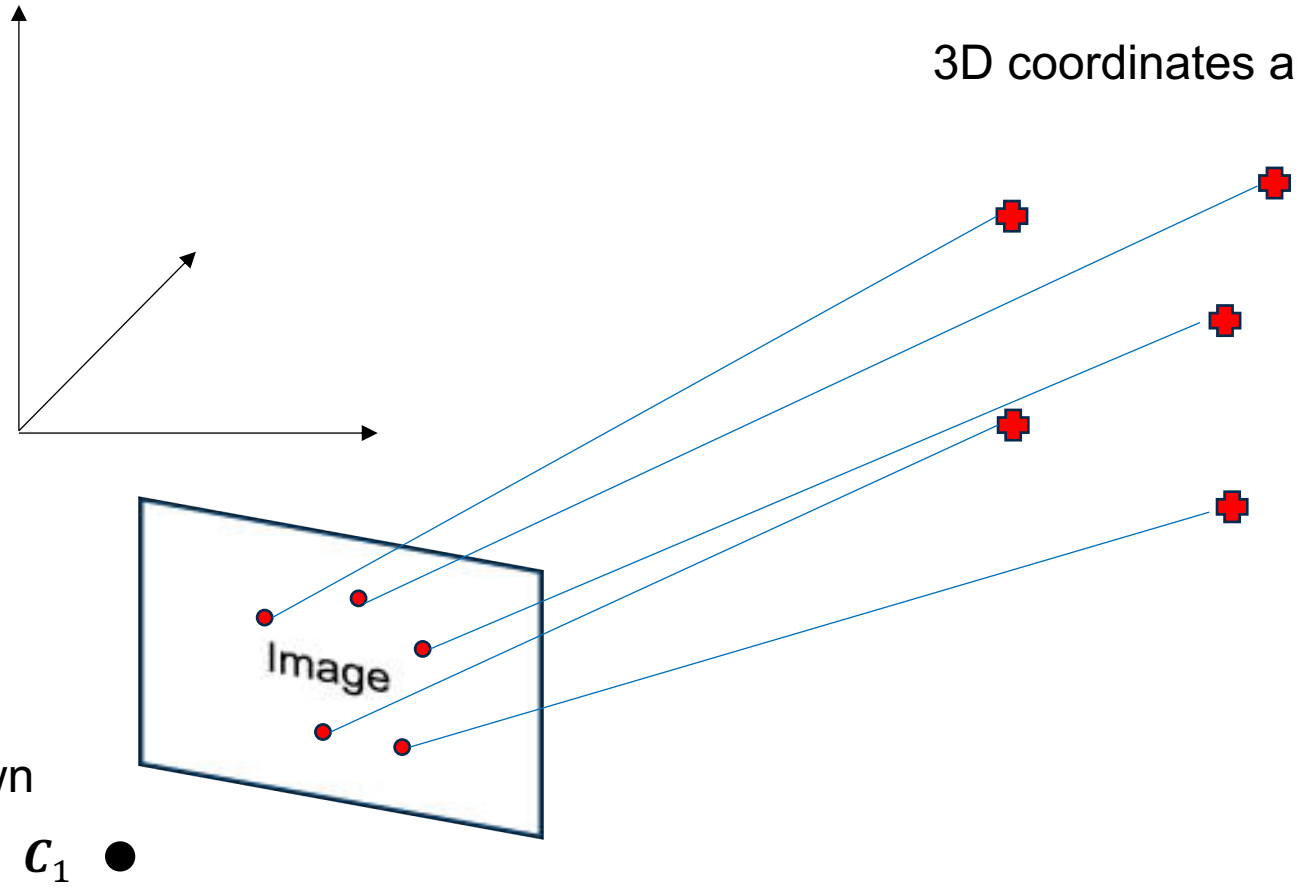


[R,t] of the camera is unknown

CRICOS No.00213J

How many 3D points to we need?

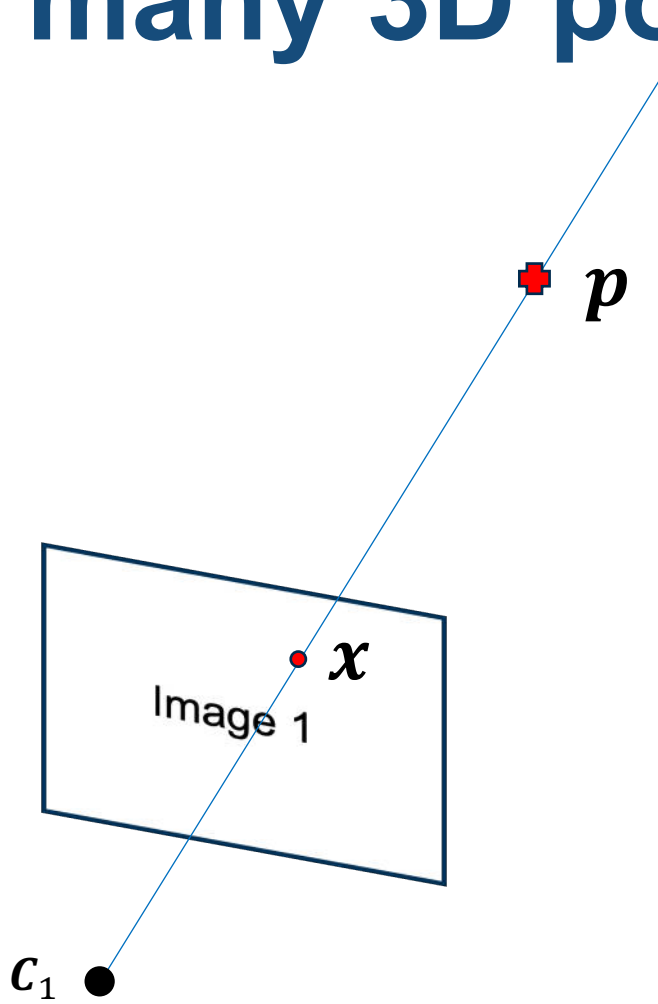
3D coordinates are known



[R,t] of the camera is unknown

CRICOS No.00213J

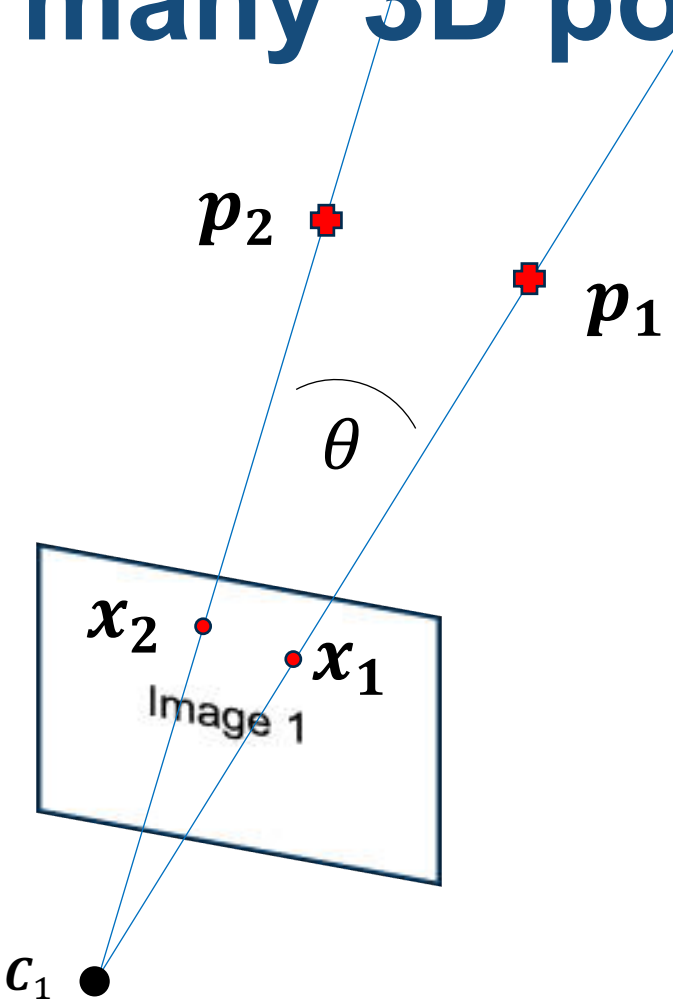
How many 3D points to we need?



1 point – infinite solutions:
Camera centre can be anywhere along the ray.

CRICOS No.00213J

How many 3D points to we need?



2 points – infinite solutions, but bounded:
Camera centre can be anywhere where angle θ is maintained.

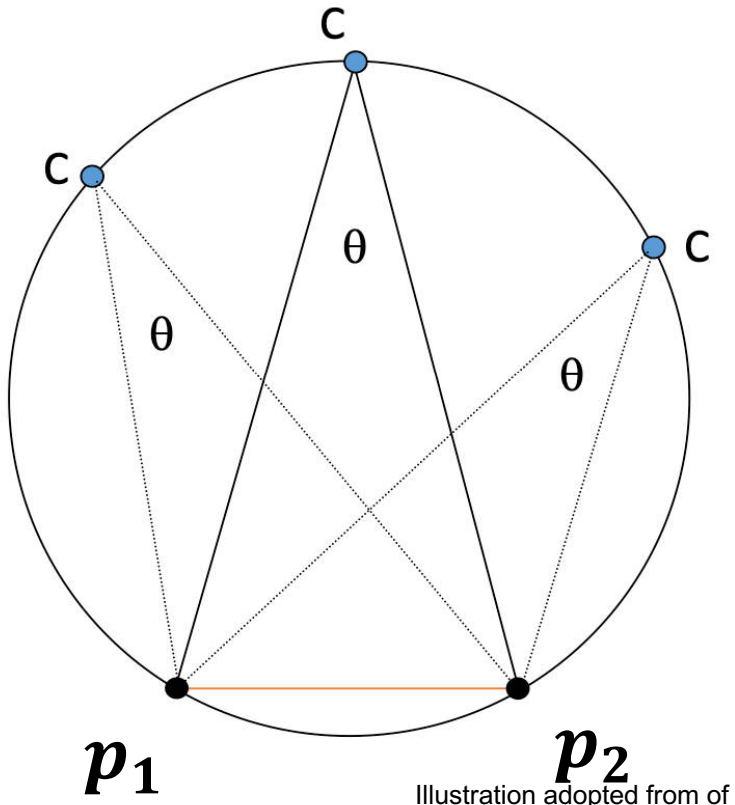
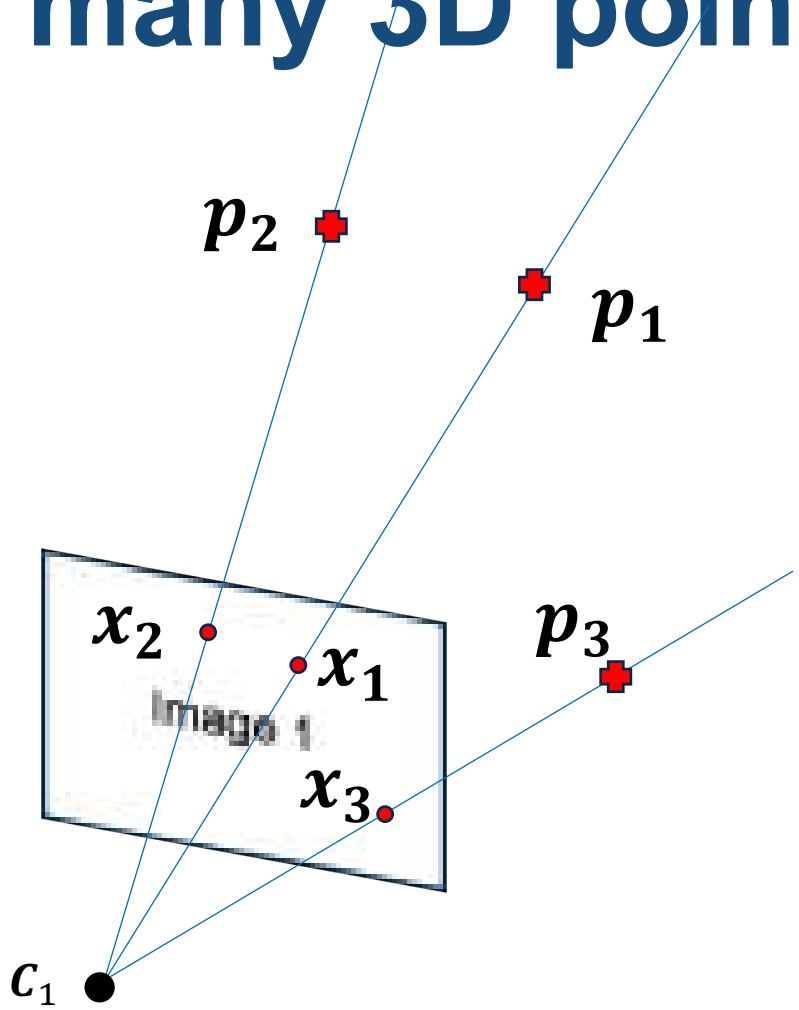


Illustration adopted from of Davide Scaramuzza

CRICOS No.00213J

How many 3D points to we need?



3 points – up to 4 solutions for R,t.

Use a 4th point to disambiguate.

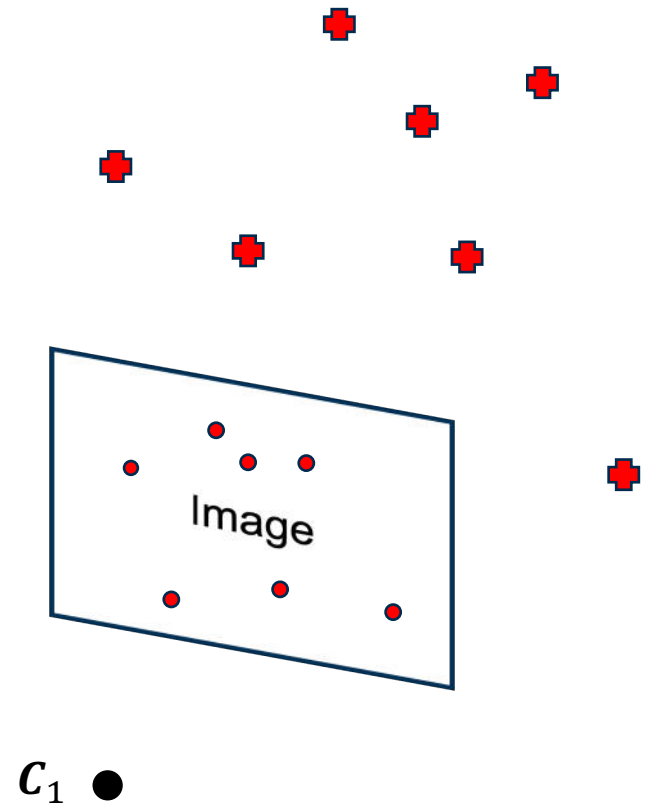
This is called the **P3P problem**
= Perspective-3-Point problem

Use OpenCV's `solvePnP()` function with `flags=SOLVEPNP_AP3P`

Algorithm from: *Ke, Roumeliotis. An Efficient Algebraic Solution to the Perspective-Three-Point Problem. CVPR'17*.

CRICOS No.00213J

How many 3D points to we need?



4 or more points

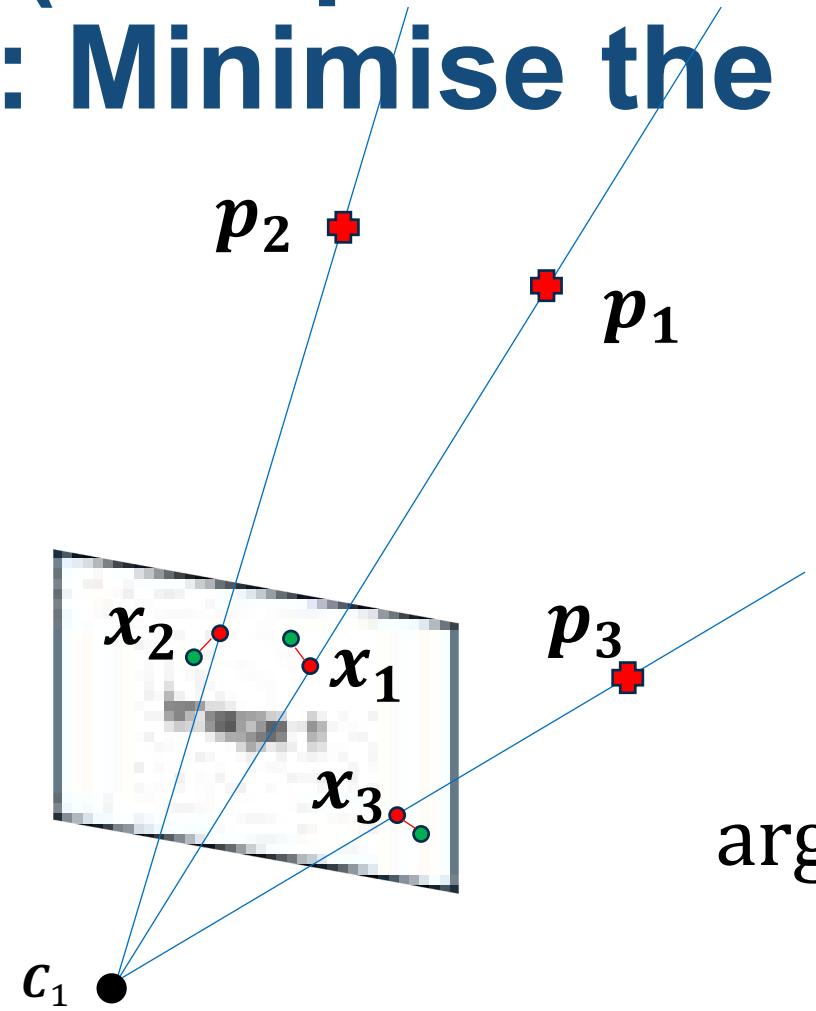
Efficient recent solution from
Lepetit, Moreno Noguer, Fua, EPnP: An Accurate O(n) Solution to the PnP Problem, International Journal of Computer Vision, 2009.

This is called the **PnP problem**
= Perspective-n-Point problem

Use OpenCV's `solvePnP()` function with `flags=SOLVEPNP_EPnP`

CRICOS No.00213J

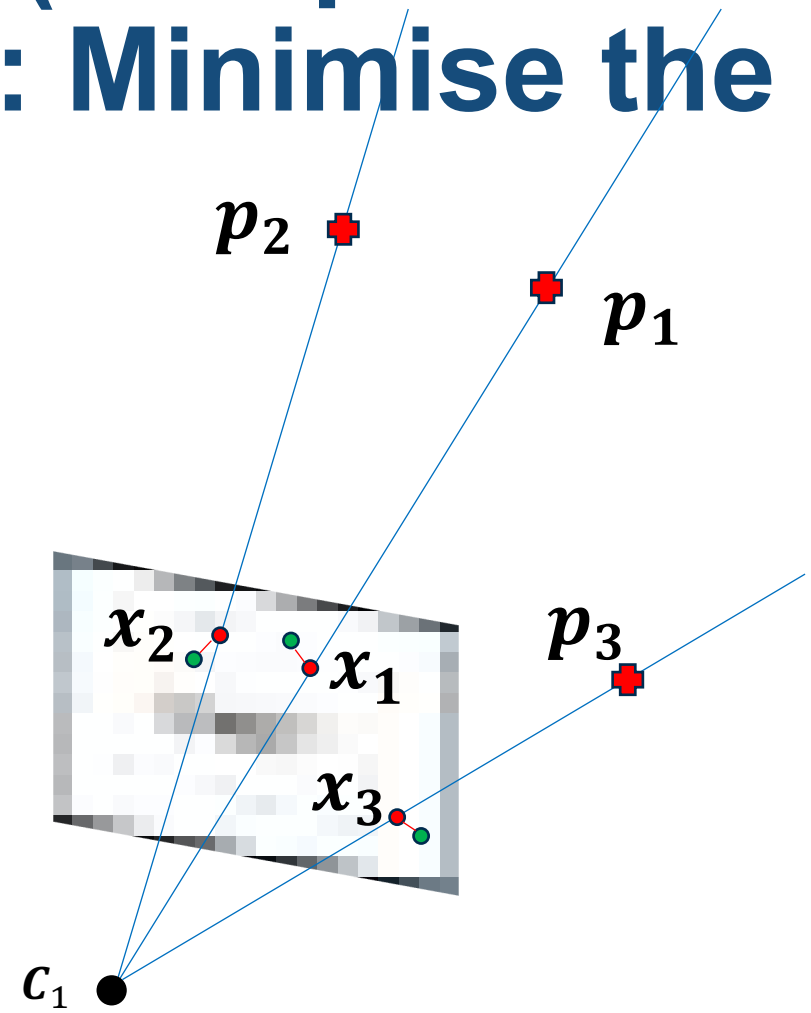
PnP (Perspective–N–Point Problem)
Idea: Minimise the Reprojection Error



$$\text{argmin}_{R,t} \sum_i \left\| x_\tau^i - K(R \cdot p^i{}_{\tau-1} + t) \right\|^2$$

CRICOS No.00213J

PnP (Perspective–N–Point Problem)
Idea: Minimise the Reprojection Error



Component	PnP
Parameters	$\boldsymbol{R}, \boldsymbol{t}$
Prediction	projected pixel $\boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^{i}{}_{\tau-1} + \boldsymbol{t})$
Residual / Error	Reprojection Error $\boldsymbol{x}^{i}_{\tau} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^{i}{}_{\tau-1} + \boldsymbol{t})$
Objective	sum of squared reprojection errors $\sum_{i} \| \cdots \|^2$

$$\mathrm{argmin}_{\boldsymbol{R},\boldsymbol{t}} \sum_{i} \left\| \boldsymbol{x}^{i}_{\tau} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}^{i}{}_{\tau-1} + \boldsymbol{t}) \right\|^2$$

CRICOS No.00213J

A Practical Pipeline

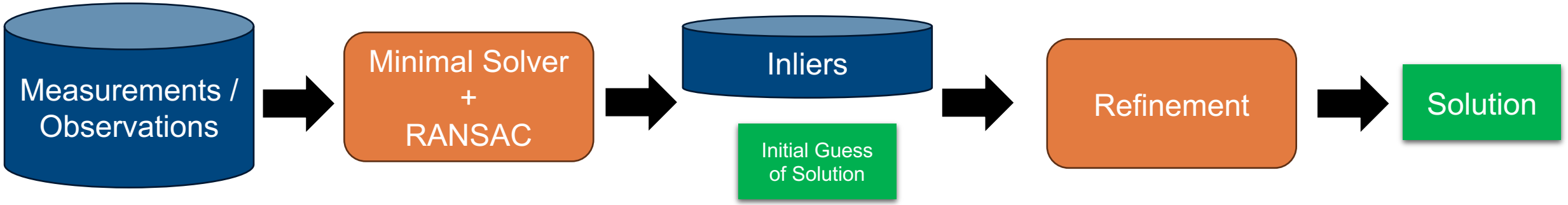
- **P3P / AP3P** can solve the PnP problem from 3 point correspondences
  - 3 correspondences is the minimal set we encountered in the context of RANSAC
  - Use P3P / AP3P in a RANSAC loop to filter outlier correspondences and get an initial estimate for rotation and translation

- **EPnP** can quickly generate a solution from many correspondences
  - Using more observations is usually better than the minimal set!
  - Downside: it is vulnerable to outliers

- **Iterative Refinement**: given an initial solution, refine it further by minimising the Reprojection Error over all point correspondences
  - vulnerable to outliers

CRICOS No.00213J

A very typical pipeline in Computer Vision



- Typically a lot of data but contains outliners
- *e.g. matched features*

Identifies outliers

- Reduced set of inliers, guaranteed* to be free of outliers
- Solution based only on minimal set
- *e.g. 8 point pairs for F, 3 point pairs for P3P*

- Typically iterative (nonlinear) Least Squares
- Requires initial guess
- Very vulnerable to outliers

CRICOS No.00213J

A Practical Pipeline

Method family	Main role	Uses	Directly minimises reprojection error?	Outlier vulnerability
P3P / AP3P	Minimal pose-hypothesis generator, typically inside RANSAC	3 correspondences mathematically; OpenCV interface commonly uses 4	**No.** Solves exact geometric/algebraic constraints; reprojection error scores the resulting hypotheses	**High alone. Robust inside RANSAC**, provided enough correct matches exist
EPnP	Fast, non-iterative initial pose estimate using many correspondences	>4 correspondences	**No.** Uses an algebraic formulation; refinement may follow	**High.** Ordinary EPnP treats incorrect matches as valid constraints
Iterative LM / GN	Refines an initial pose using all selected correspondences	Initial pose plus typically many correspondences	**Yes.** Directly minimises total reprojection error	**High with ordinary least squares.** Use RANSAC inliers and optionally a robust loss

P3P/AP3P proposes robust hypotheses, EPnP provides a fast all-point estimate, and LM/GN provides the final reprojection-error refinement.

CRICOS No.00213J

A Practical Pipeline (OpenCV Functions)

Method family	Direct function/flag	Robust RANSAC access	Explicit refinement
P3P	solvePnP(..., SOLVEPNP_P3P)	solvePnPRansac(..., SOLVEPNP_P3P)	—
AP3P	solvePnP(..., SOLVEPNP_AP3P)	solvePnPRansac(..., SOLVEPNP_AP3P)	—
EPnP	solvePnP(..., SOLVEPNP_EPNP)	solvePnPRansac(..., SOLVEPNP_EPNP)	—
Iterative LM	solvePnP(..., SOLVEPNP_ITERATIVE)	solvePnPRansac(..., SOLVEPNP_ITERATIVE)	solvePnPRefineLM(...)
Iterative GN-like	—	—	solvePnPRefineVVS(...)

CRICOS No.00213J

A Practical Pipeline (OpenCV Functions)

- First: RANSAC with minimal P3P solver to filter outliers and generate initial guess
- Then: Iterative refinement using only the inliers and the initial guess as starting point for the optimisation

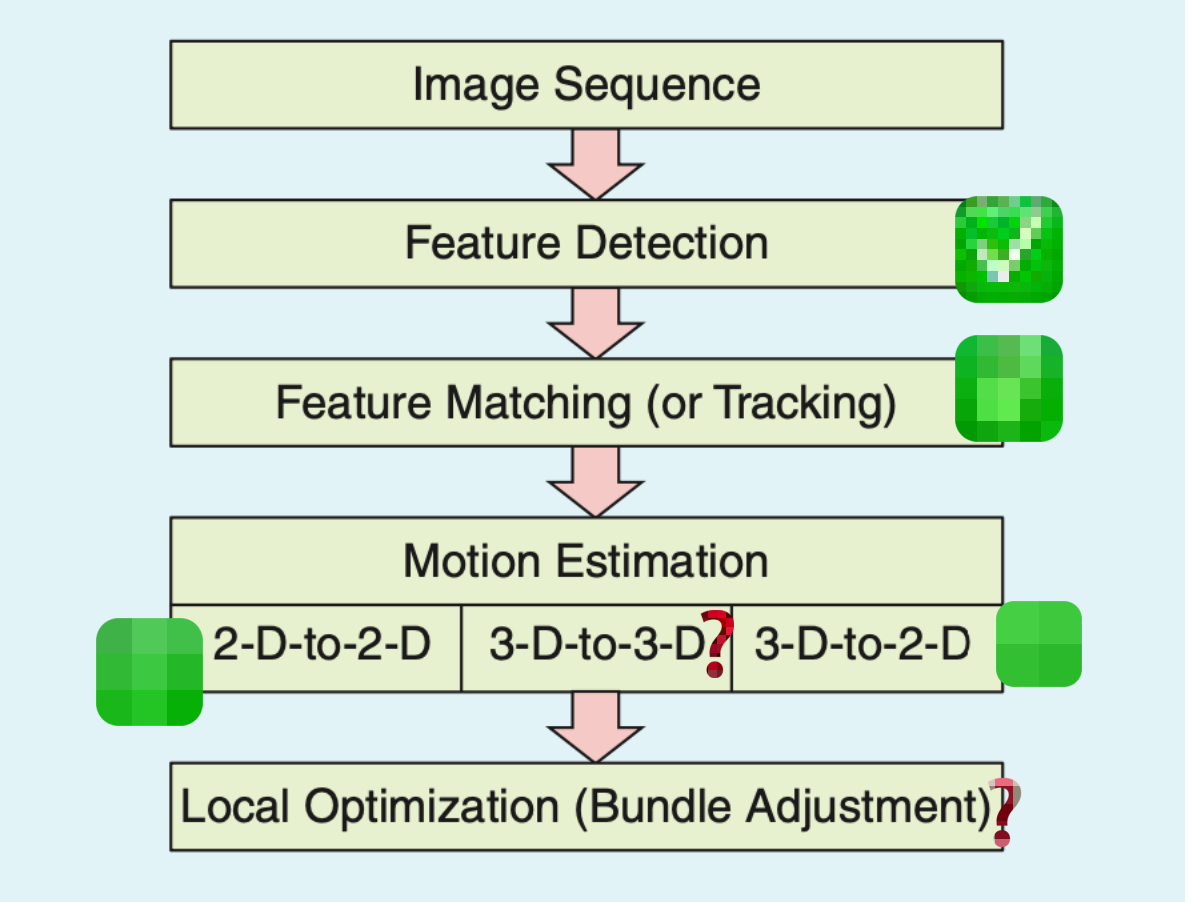
```
ok, rvec, tvec, inliers = cv2.solvePnPRansac(points3D, points2D, K, flags=cv2.SOLVEPNP_AP3P)

idx = inliers.ravel()

rvec, tvec = cv2.solvePnPRefineLM(points3D[idx], points2D[idx], K, rvec, tvec)
```

CRICOS No.00213J

Recap – What we learned so far



Let's think and Discuss:

How would you build a Visual Odometry Pipeline using what we learned so far?



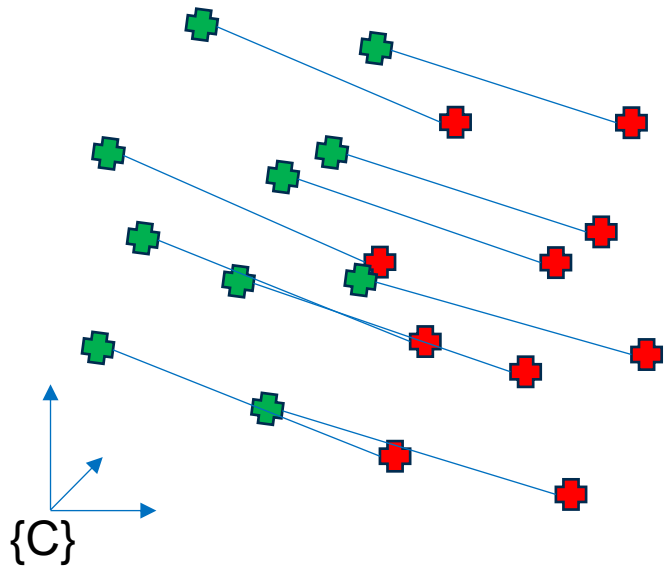
CRICOS No.00213J



Camera motion from 3D-3D Correspondences

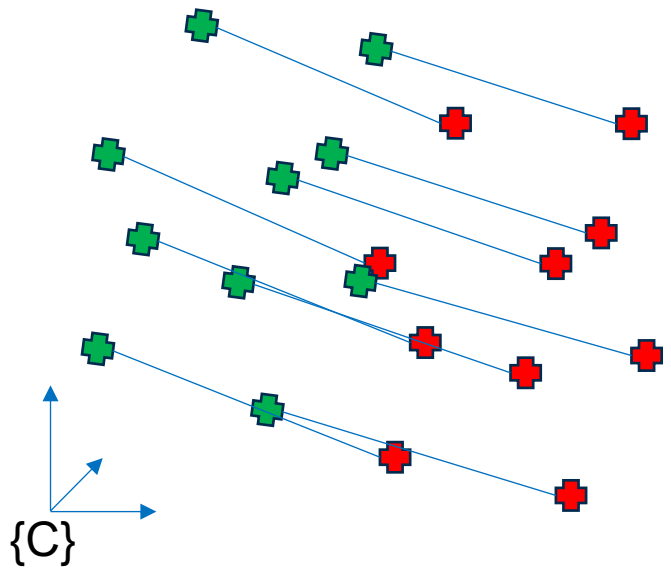
3D-3D Correspondences

- Estimate camera motion from two sets of 3D points



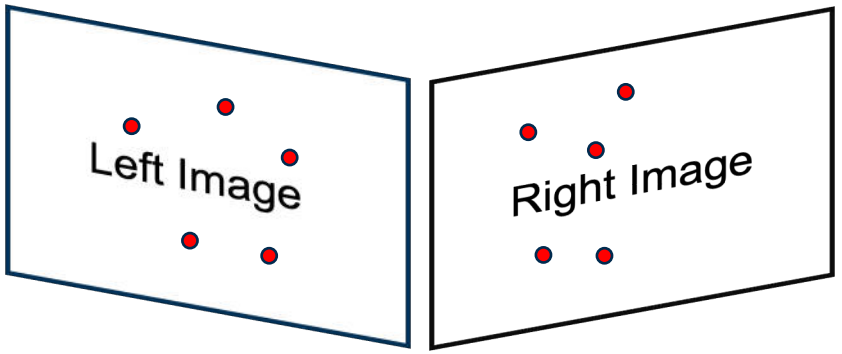
3D-3D Correspondences

- Estimate camera motion from two sets of 3D points
- How can we obtain 3D coordinates for points?
  - Stereo / RGBD cameras
  - Observe known objects
  - Fusion with other sensors

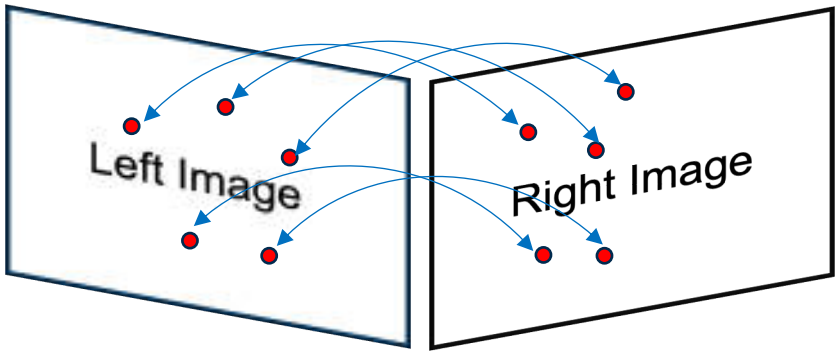


CRICOS No.00213J

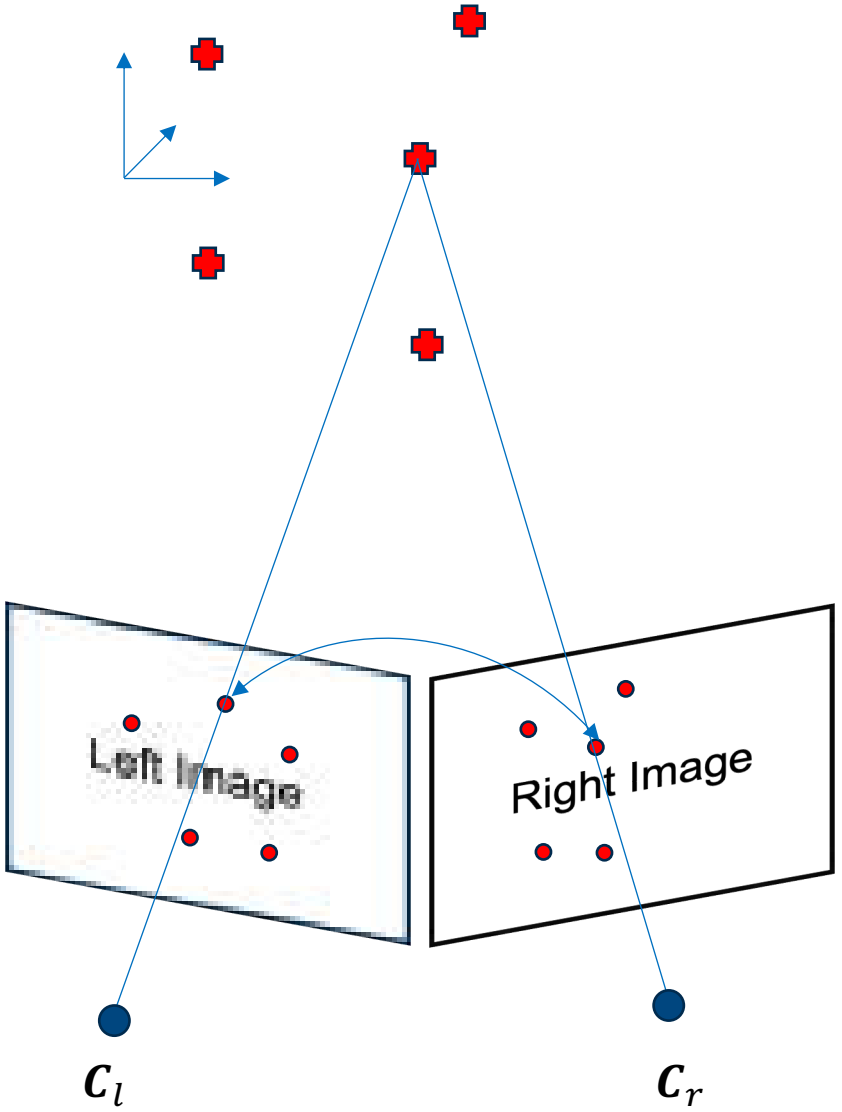
t=0



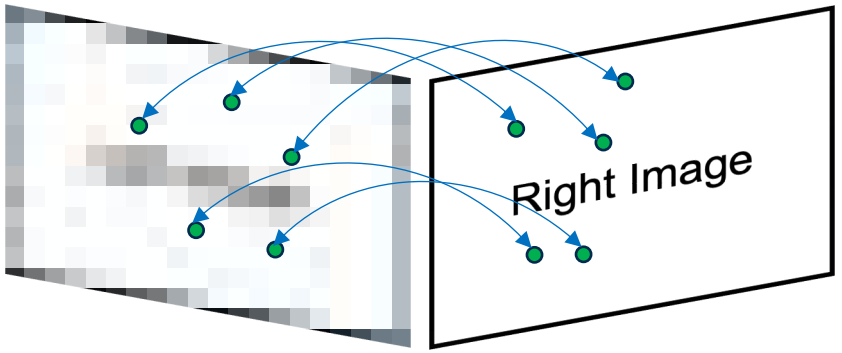
t=0



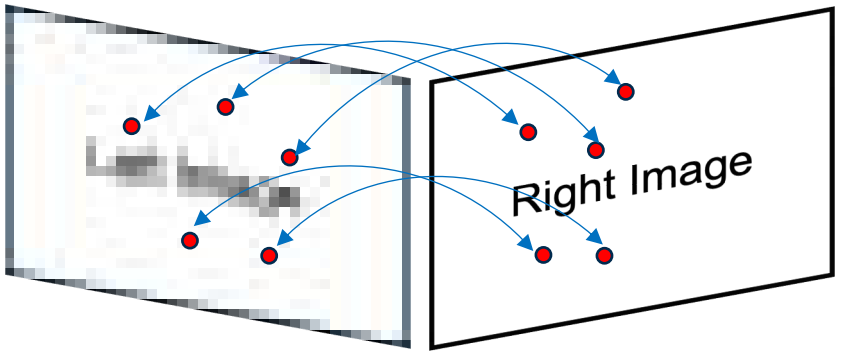
t=0

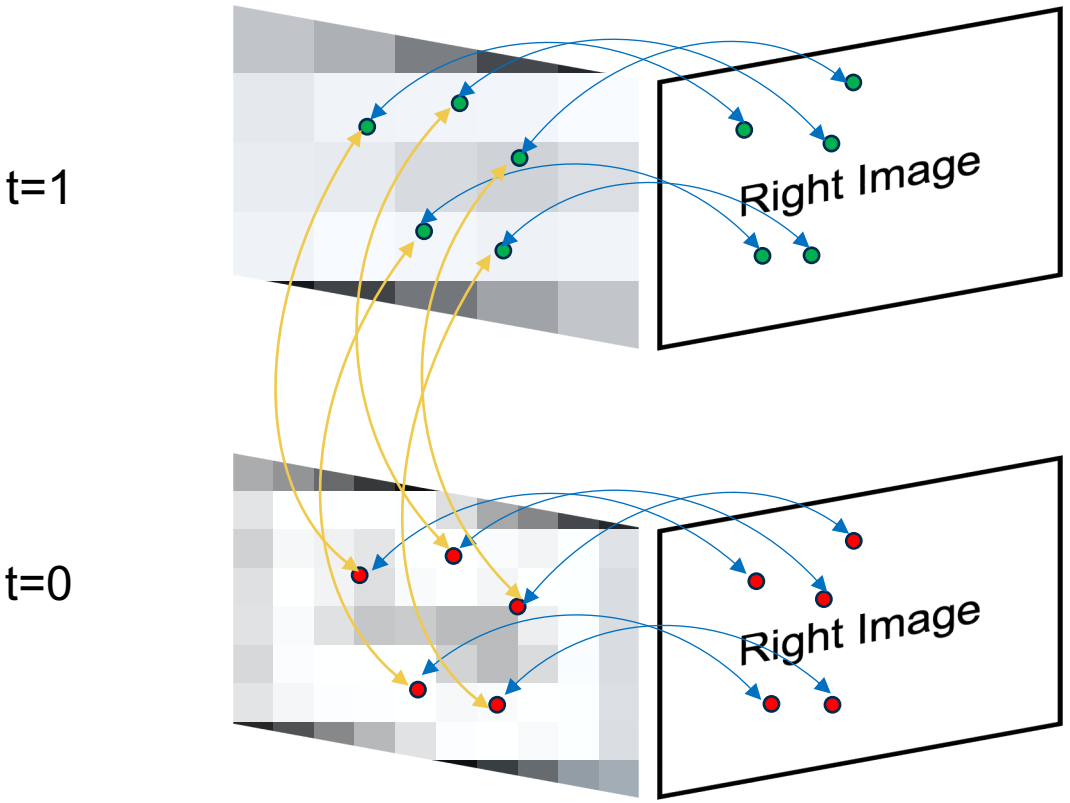


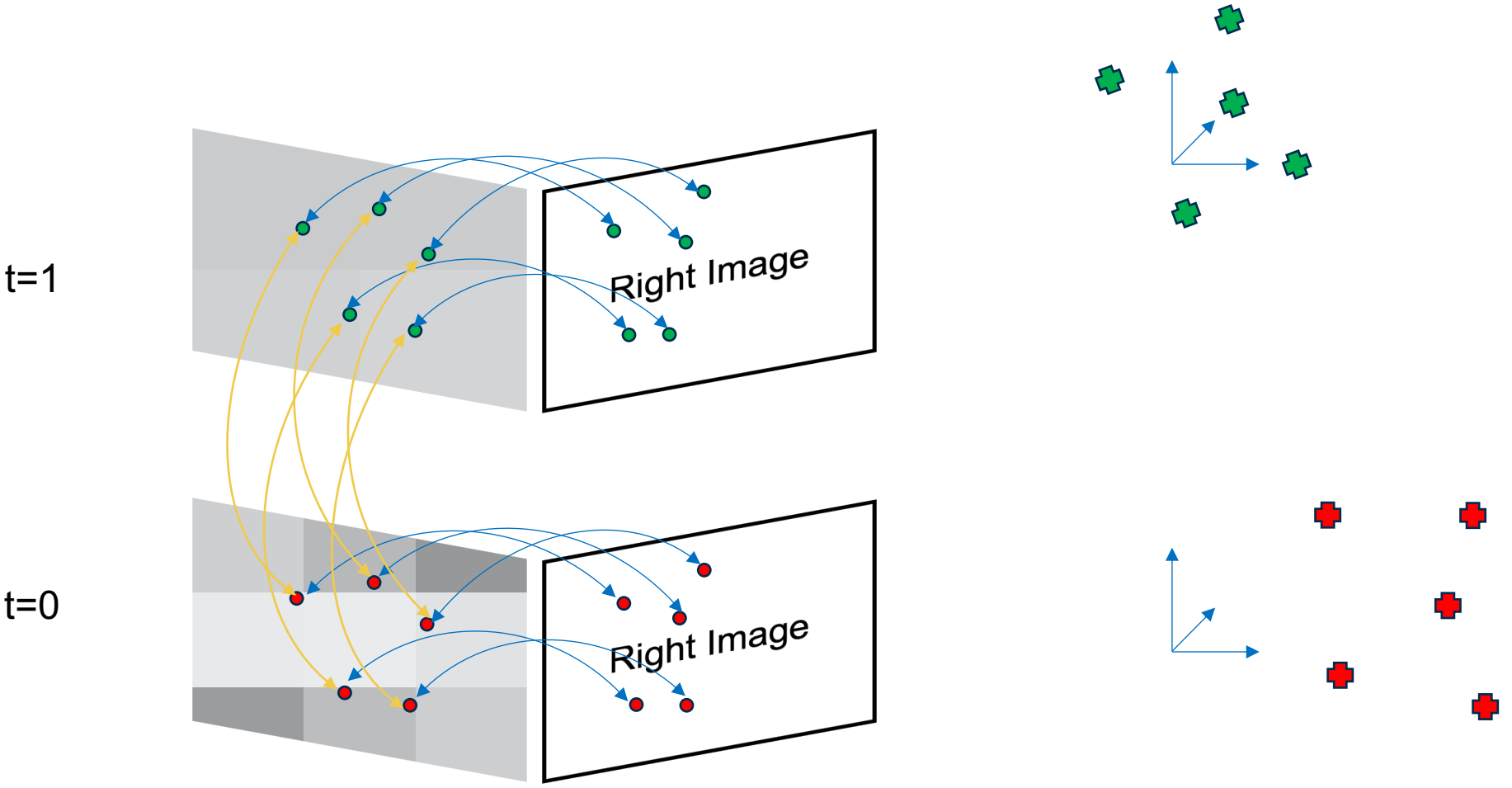
t=1

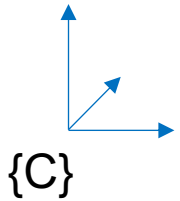


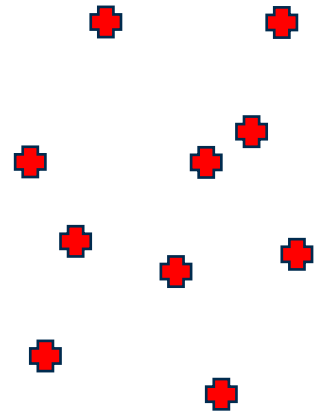
t=0

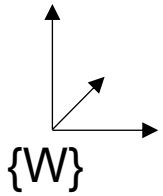


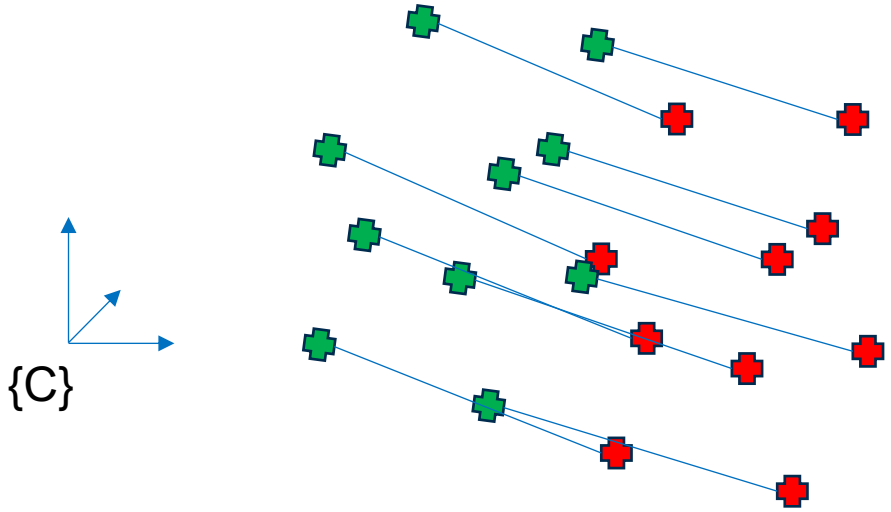

t=1
Right Image
t=0
Right Image

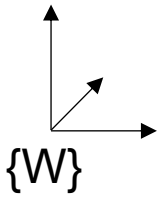

t=1
Right Image
t=0
Right Image


{C}

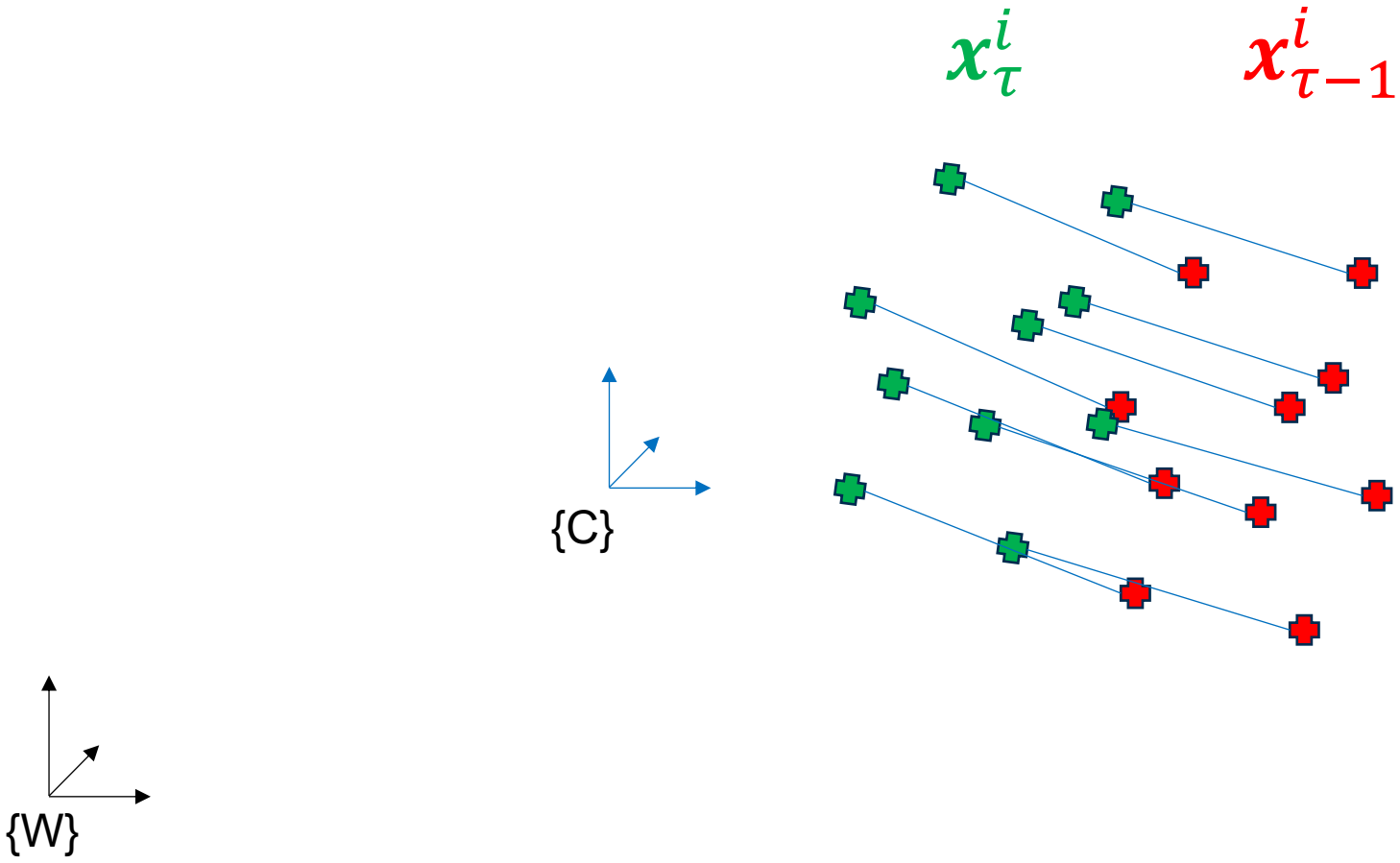



{W}

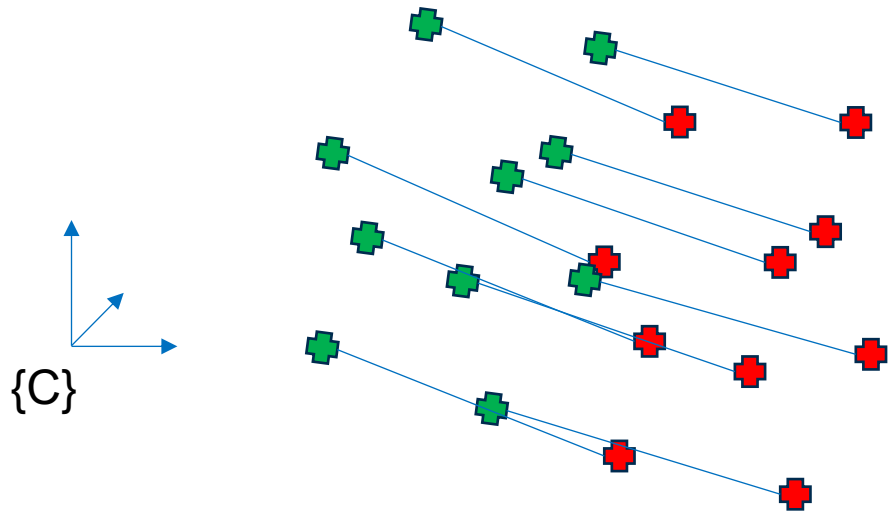


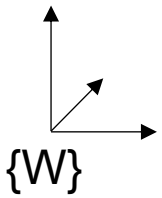


CRICOS No.00213J

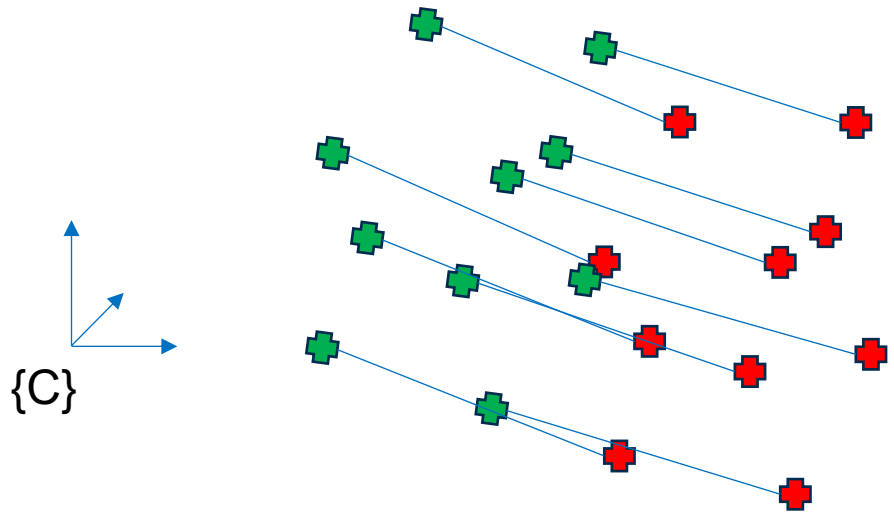

x_τ^i
$x_{\tau-1}^i$
{C}
{W}

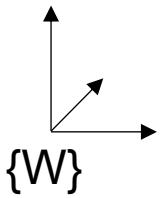
$$x_\tau^i = T \cdot x_{\tau-1}^i$$





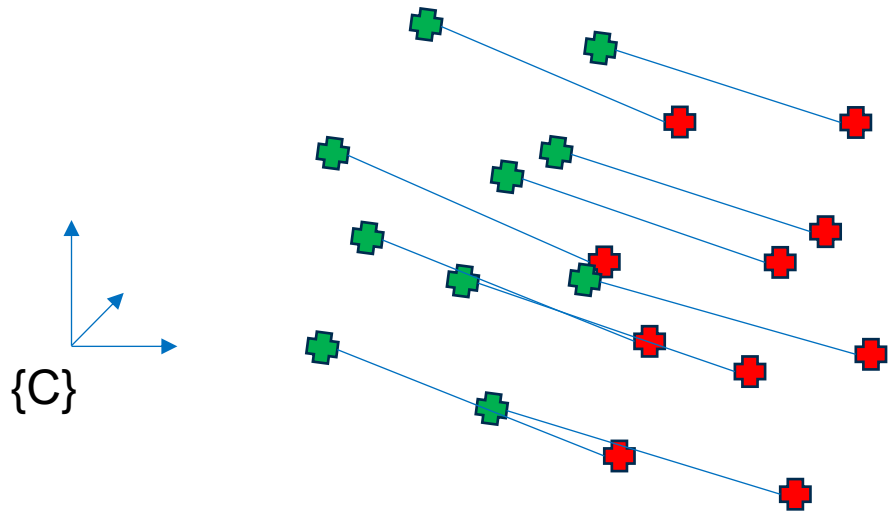
$$x_\tau^i = R \cdot x_{\tau-1}^i + t$$

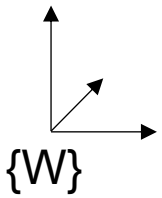




$$x_\tau^i = R \cdot x_{\tau-1}^i + t$$

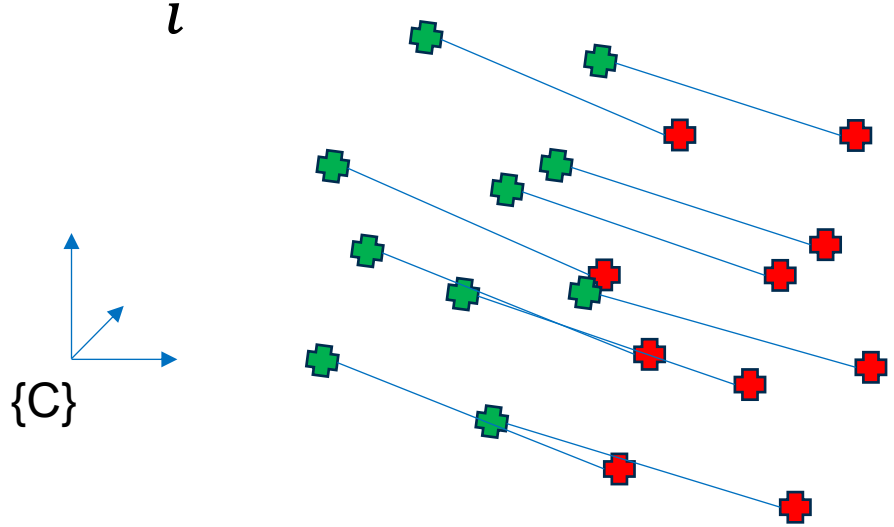
R and **t** are unknown, how to solve for them?





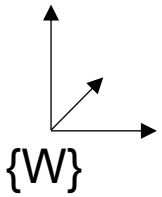
$$\text{argmin}_{\boldsymbol{R},\boldsymbol{t}} \sum_{i} \left\| \boldsymbol{x}^{i}{}_{\tau} - (\boldsymbol{R} \cdot \boldsymbol{x}^{i}{}_{\tau-1} + \boldsymbol{t}) \right\|^{2}$$

R and **t** are unknown, how to solve for them?



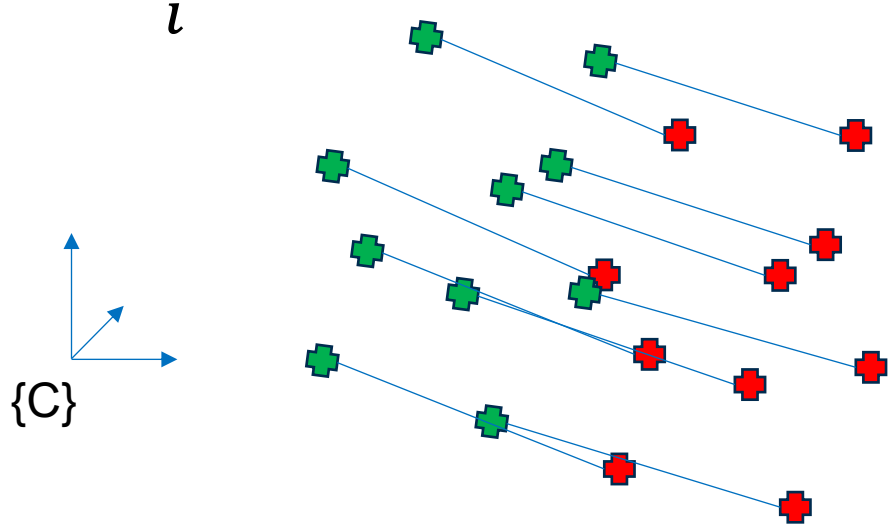
Idea:

- Find the **R** and **t** that mininise the Euclidean distance between both sets of corresponding 3D points.



$$\text{argmin}_{\boldsymbol{R},\boldsymbol{t}} \sum_{i} \left\| \boldsymbol{x}^{i}{}_{\tau} - (\boldsymbol{R} \cdot \boldsymbol{x}^{i}{}_{\tau-1} + \boldsymbol{t}) \right\|^{2}$$

R and **t** are unknown, how to solve for them?

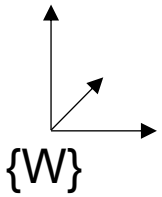


Idea:

- Find the **R** and **t** that mininise the Euclidean distance between both sets of corresponding 3D points.

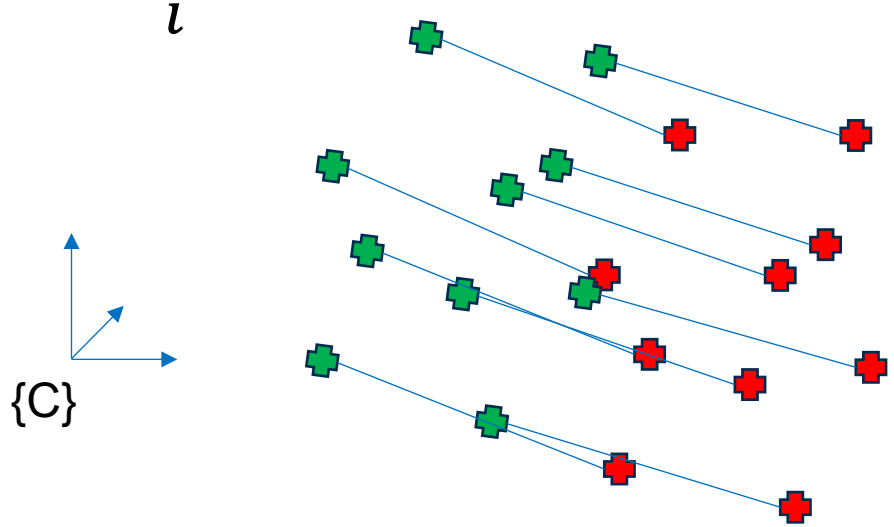
Question:

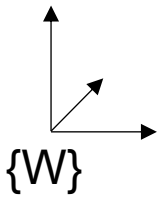
- Would you expect the sum to be zero with the true **R** and **t**?



$$\text{argmin}_{R,t} \sum_i \left\| x^i{}_\tau - (R \cdot x^i{}_{\tau-1} + t) \right\|^2$$

R and **t** are unknown, how to solve for them?





Idea:

- Find the **R** and **t** that mininise the Euclidean distance between both sets of corresponding 3D points.

How to solve this problem?

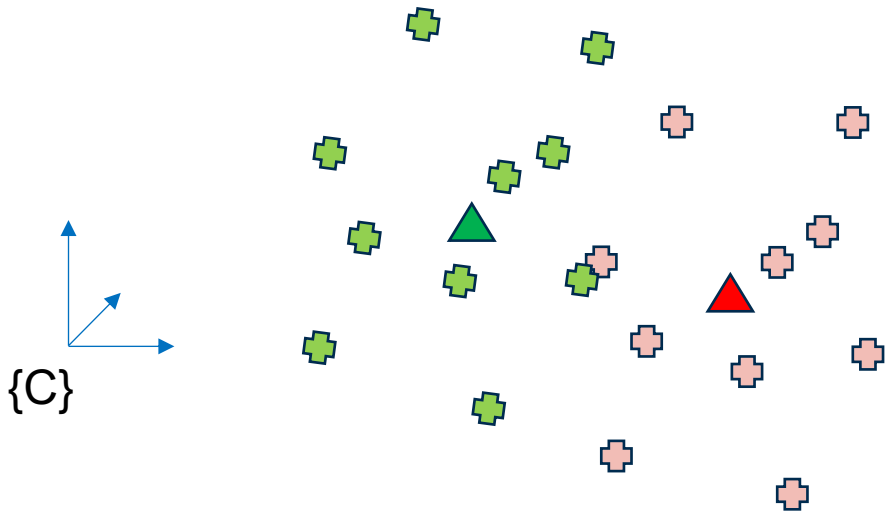
Idea:

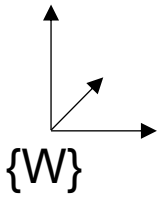
- Decouple estimation of translation **t** from rotation **R**
- Estimate both separately.

- Published by K. S. Arun, T. S. Huang and S. D. Blostein, "Least-Squares Fitting of Two 3-D Point Sets," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1987.

CRICOS No.00213J

Step 1: Find the centroids of both sets.

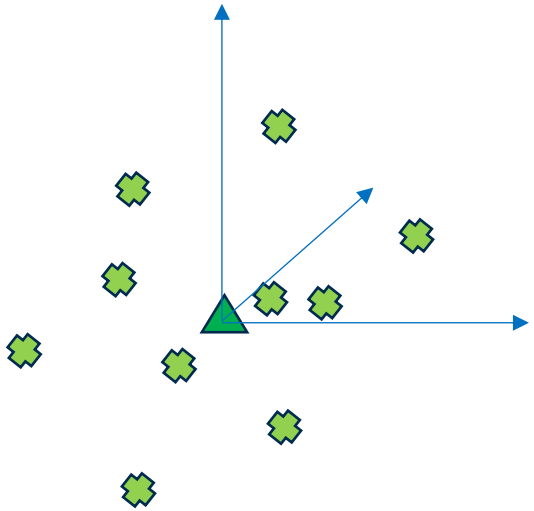




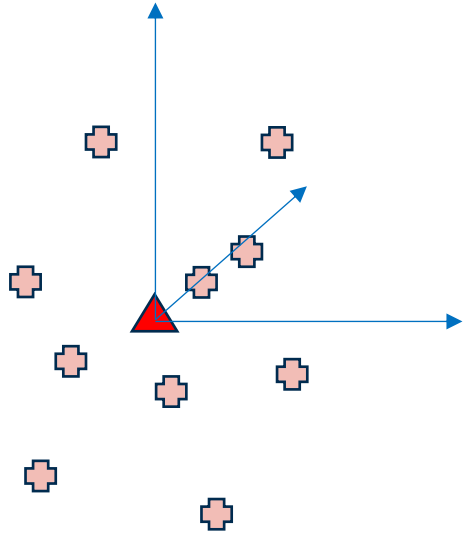
Centroids of both sets of 3D points

Step 2: Subtract the centroids from 3D coordinates.

$$\overline{\boldsymbol{x}}^{i}{}_{\tau} = \boldsymbol{x}^{i}_{\tau} - \overline{\boldsymbol{x}}_{\tau}$$



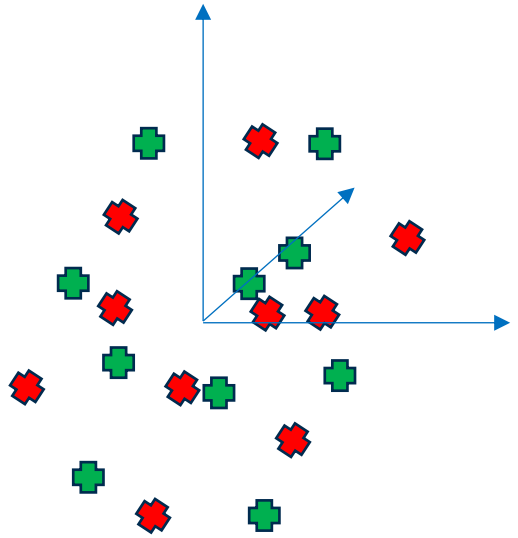
$$\overline{\boldsymbol{x}}^{i}{}_{\tau-1} = \boldsymbol{x}^{i}_{\tau-1} - \overline{\boldsymbol{x}}_{\tau-1}$$



Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

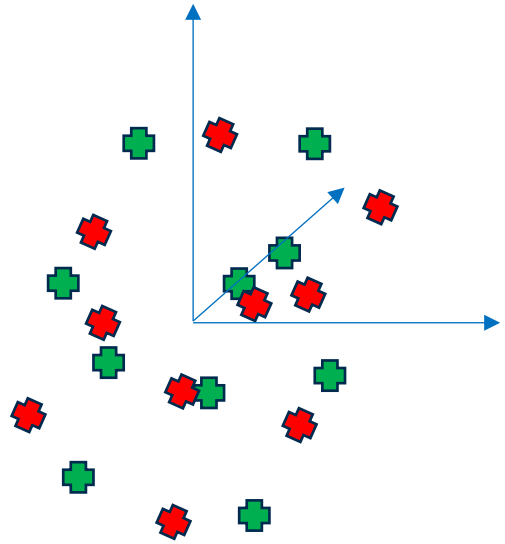
$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$



Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

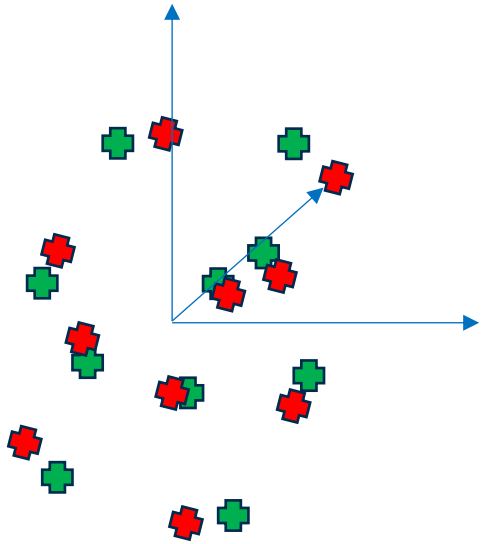
$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$



Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

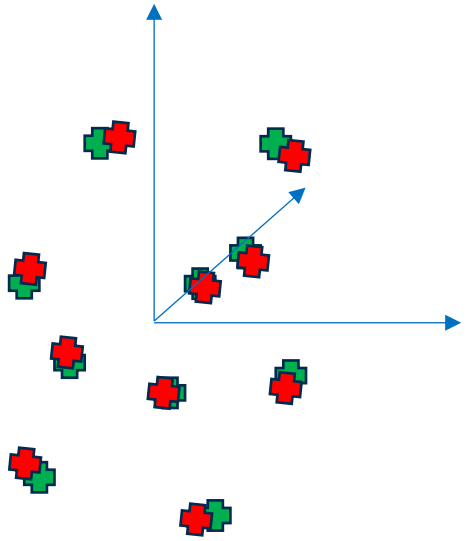
$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$



Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

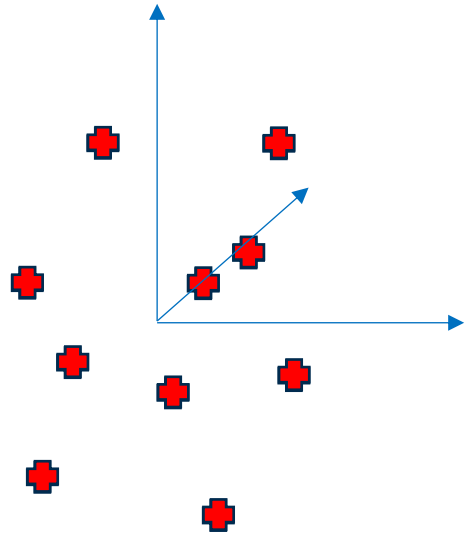
$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$



Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$



Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$

3 Steps:

$$\boldsymbol{H} = \sum_i^n \bar{x}^i_{\tau-1} \cdot \bar{x}^{iT}_{\tau}$$

$$\boldsymbol{USV}^T = \text{svd}(\boldsymbol{H})$$

$$\boldsymbol{R} = \boldsymbol{VU}^T$$

Outer product, results in 3x3 matrix.
`np.outer(a,b)`

H is the cross-covariance matrix of the matched points.

svd = singular value decomposition
`np.linalg.svd()`

But what is the Outer Product?

$$\boldsymbol{x} \otimes \boldsymbol{y} = \boldsymbol{x}\boldsymbol{v}^T = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} \begin{pmatrix} y_1 & y_2 & y_3 \end{pmatrix} = \begin{pmatrix} x_1\, y_1 & x_2 y_1 & x_3 y_1 \\ x_2 y_1 & x_2 y_2 & x_3 y_2 \\ x_3 y_1 & x_2 y_2 & x_3 y_3 \end{pmatrix}$$

- A Matrix where each element reflects pairwise product of the components of **x** and **y.**
- Pairwise products are big when components are strongly correlated.

But what is the Outer Product?

$$\boldsymbol{x} \otimes \boldsymbol{y} = \boldsymbol{x}\boldsymbol{v}^T = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} (y_1 \quad y_2 \quad y_3) = \begin{pmatrix} x_1\, y_1 & x_2 y_1 & x_3 y_1 \\ x_2 y_1 & x_2 y_2 & x_3 y_2 \\ x_3 y_1 & x_2 y_2 & x_3 y_3 \end{pmatrix}$$

- A Matrix where each element reflects pairwise product of the components of **x** and **y.**
- Pairwise products are big when components are strongly correlated.

$$\boldsymbol{H} = \sum_{i}^{n} \bar{x}_{\tau-1}^{i} \cdot \bar{x}_{\tau}^{i\,T}$$

- By summing the outer products over all corresponding points, the resulting matrix **H** aggregates the correlations across the entire set of points.
  - Entries on the diagonal: correlation along the same axis.
  - Off-diagonal: cross-axis correlation, capturing rotation information.

CRICOS No.00213J

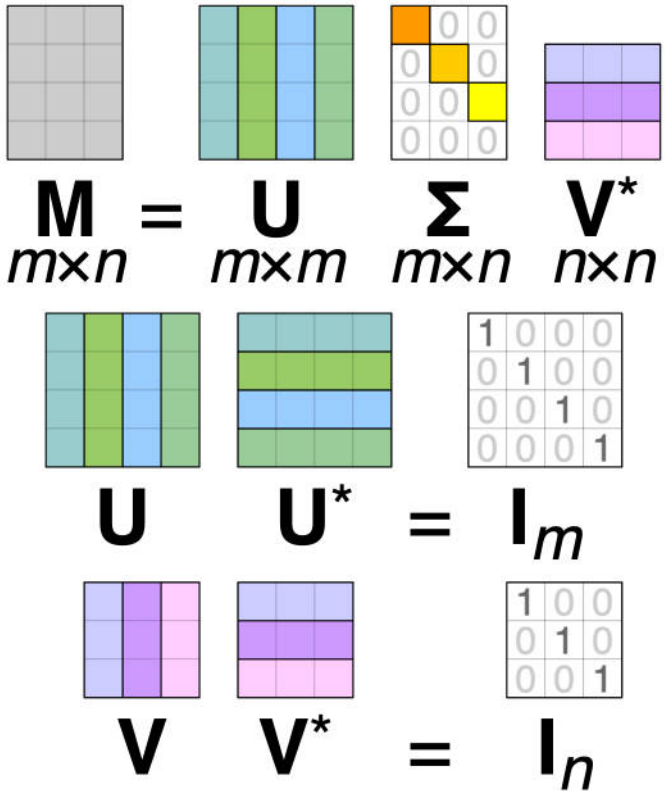
But what is the SVD?

SVD decomposes a matrix **M** into three other matrices:

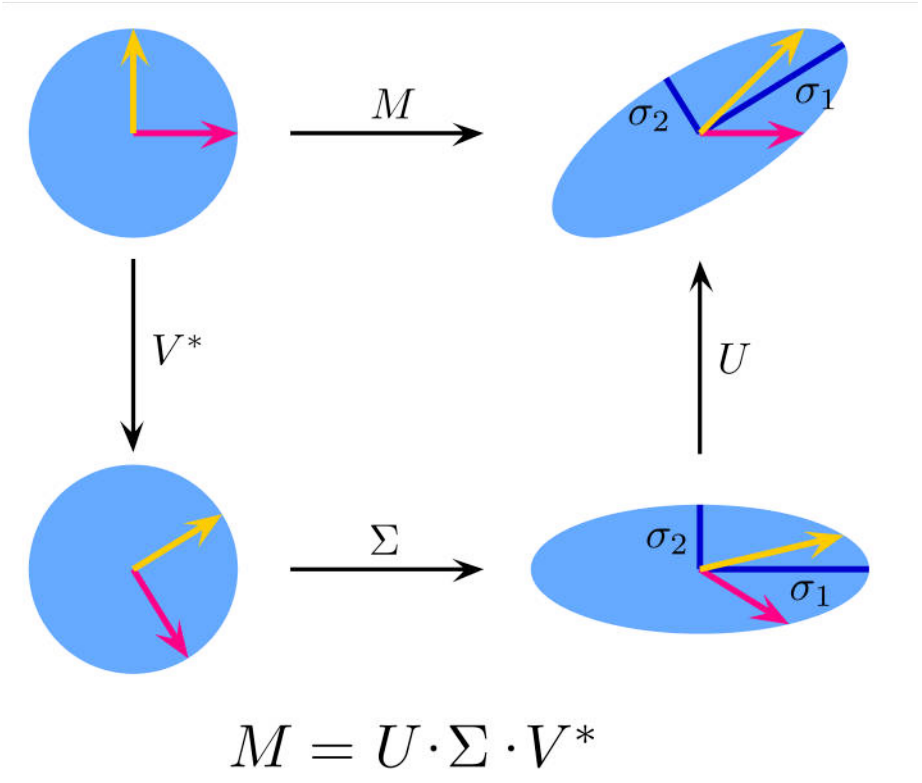
- a rotation $\boldsymbol{U}$
- a diagonal matrix $\boldsymbol{\Sigma}$
- another rotation $\boldsymbol{V}$

- $\boldsymbol{\Sigma}$ contains the Eigenvalues

$$\boldsymbol{M = U\Sigma V^T}$$



By Cmglee - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=67853297



By Georg-Johann - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=11342212

Step 3: Find the optimal rotation.

Find the rotation R that minimises the Euclidean distance between corresponding 3D points.

$$\text{argmin}_R \sum_i \left\| \bar{x}^i{}_\tau - R \cdot \bar{x}^i{}_{\tau-1} \right\|^2$$

3 Steps:

$$\boldsymbol{H} = \sum_i^n \bar{x}^i_{\tau-1} \cdot \bar{x}^{i\,T}_{\tau}$$

$$\boldsymbol{USV}^T = \text{svd}(\boldsymbol{H})$$

$$\boldsymbol{R} = \boldsymbol{VU}^T$$

The proof that this actually works is in the paper.

(K. S. Arun, T. S. Huang and S. D. Blostein, "Least-Squares Fitting of Two 3-D Point Sets," in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1987.)

There are potential degenerate cases where the solution is not valid.

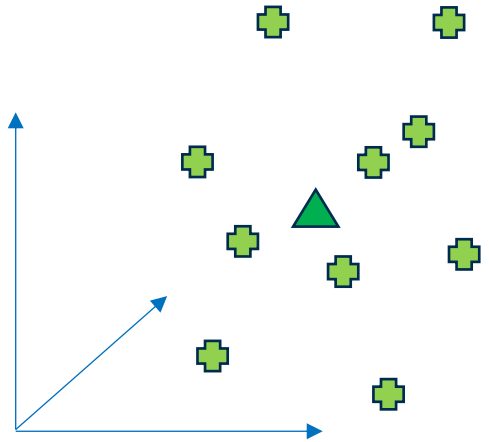
Have a look at the paper to find out:

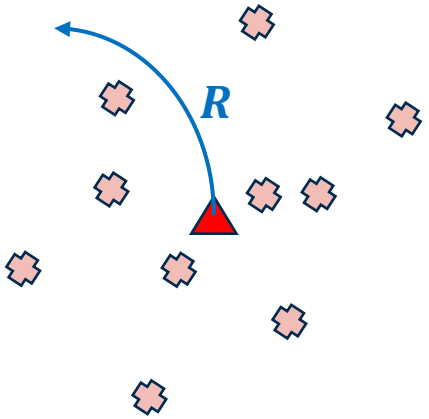
a) Which degenerate cases exist.
b) How they can be detected.
c) How they can be mitigated.

Step 4: Find the optimal translation.

Now we have **R**, but what is **t**?

$$t = \overline{x}_{\tau} - R \cdot \overline{x}_{\tau-1}$$

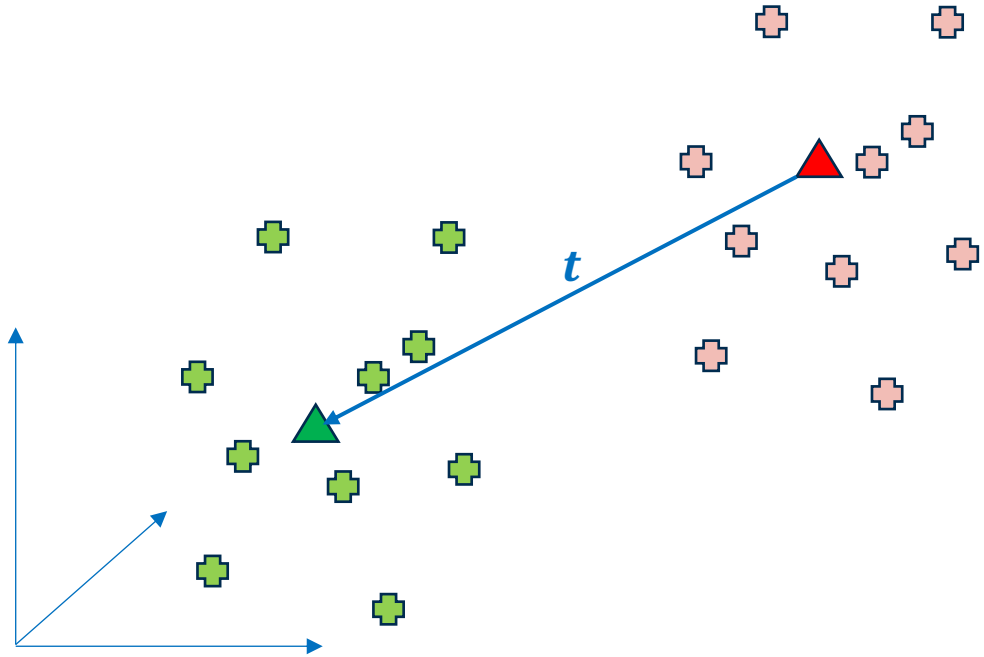




Step 4: Find the optimal translation.

Now we have **R**, but what is **t**?

$$\boldsymbol{t} = \overline{\boldsymbol{x}}_{\tau} - \boldsymbol{R} \cdot \overline{\boldsymbol{x}}_{\tau-1}$$

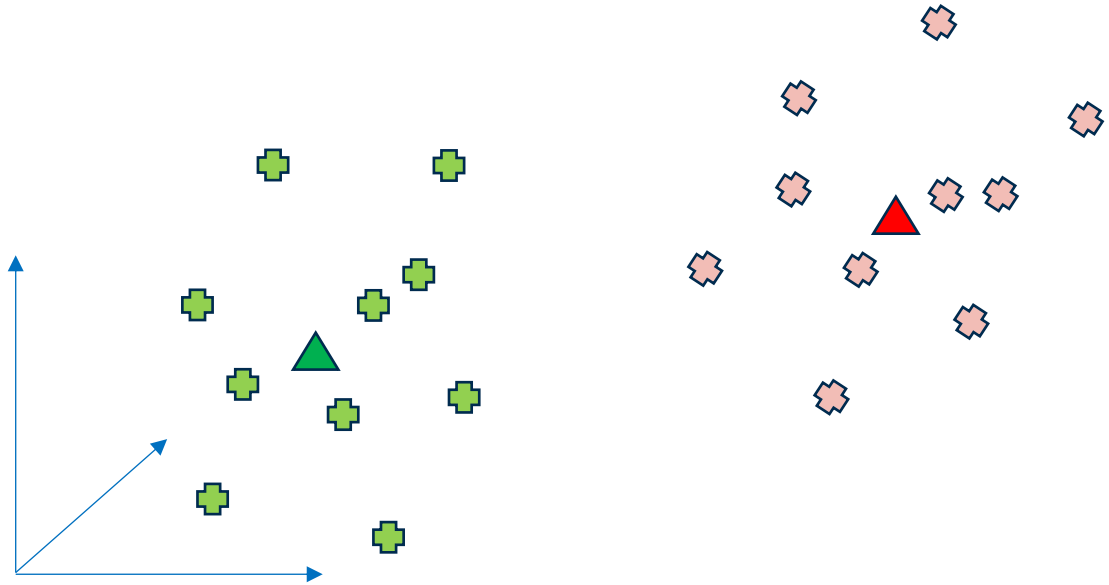


Camera motion from **R** and **t**

Now we have R and **t**, that align both sets of 3D points.

The camera motion will be the inverse!

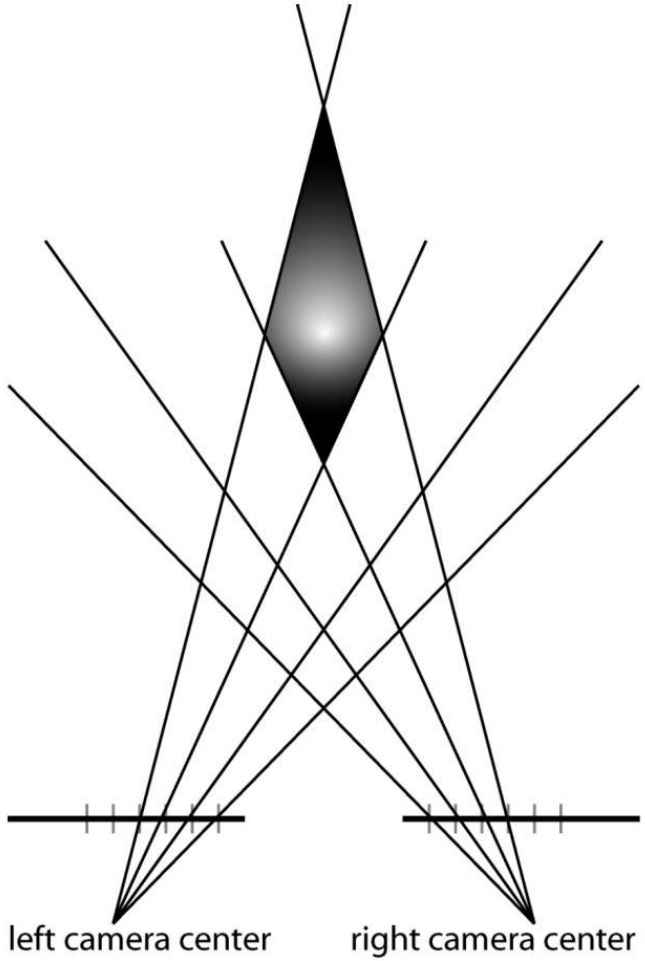
$$x_{\tau}^{i} = R \cdot x_{\tau-1}^{i} + t$$



CRICOS No.00213J

Not very accurate

- 3D-3D methods minimise the Euclidean distances between 3D points
- But 3D points from triangulation / stereo will have high depth error.
- Idea: estimate uncertainty of 3D coordinates, take them into account when solving the minimization problem. [1]
- Generally it is better to minimize image-level quantities such as reprojection error (see 2D-2D and 3D-2D based methods).

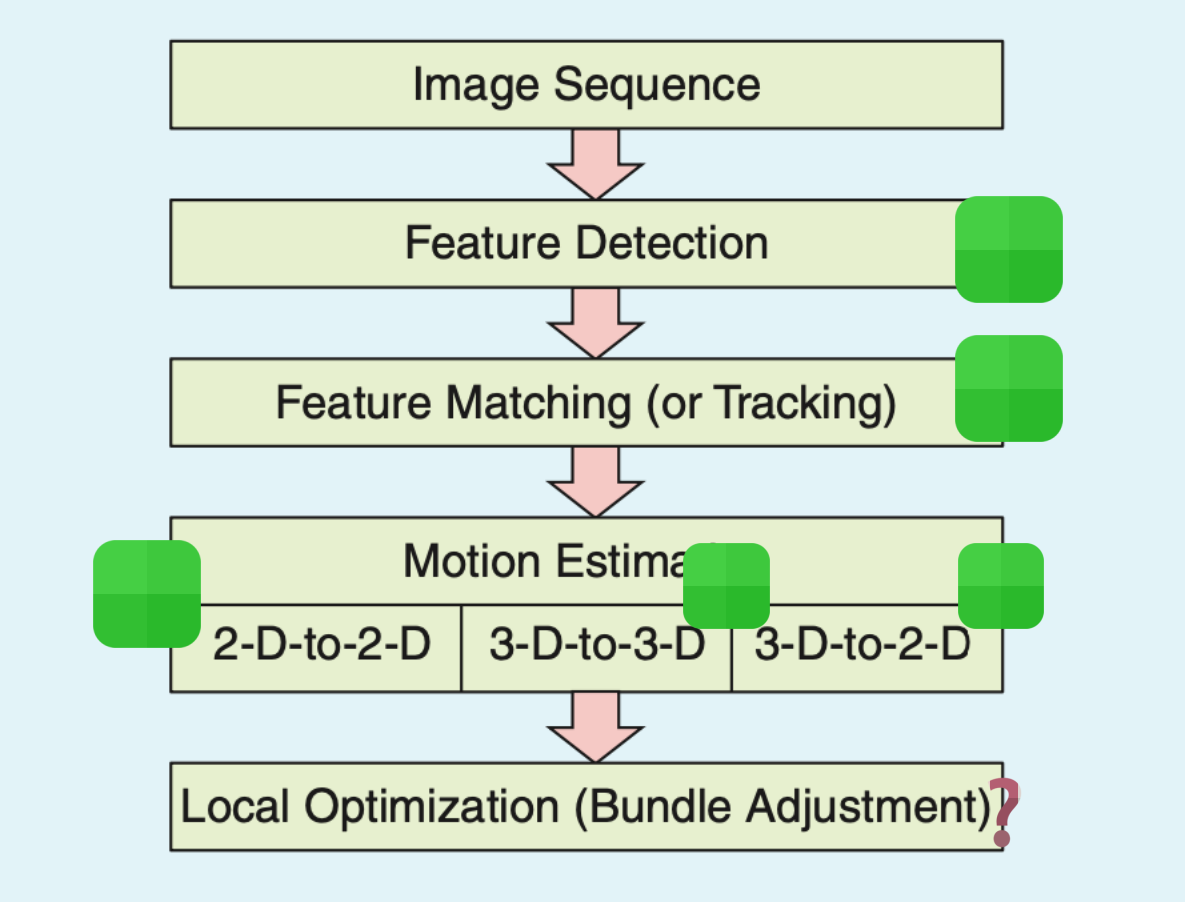


The triangulated world point might be anywhere inside the shaded diamond area.

[1] C. Olson, L. Matthies, M. Schoppers, and Maimone Maimone. Robust stereo ego-motion for long distance navigation. In Conference on Computer Vision and Pattern Recognition (CVPR), 2000.

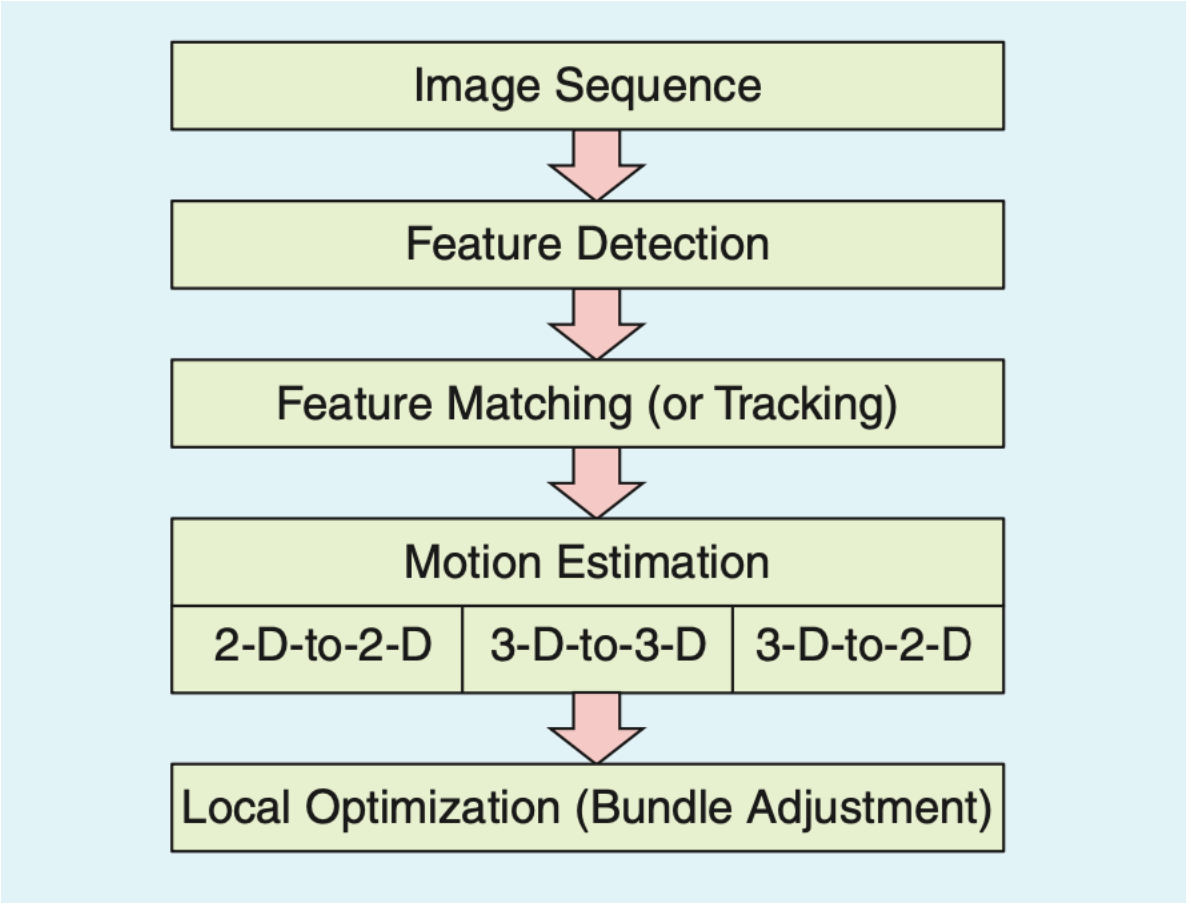
CRICOS No.00213J

Recap – What we learned so far



Recap – What we learned so far

- Now we have motion estimation methods for 2D-2D, 3D-3D and 3D-2D correspondences.
- They estimate motion between two frames, i.e. ${}^{C_t}\boldsymbol{T}_{C_{t+1}}$



CRICOS No.00213J



Keyframes

Should we use every camera frame or skip some?

- Imagine your favourite visual odometry method estimates the frame-to-frame motion of the camera: ${}^{C_t}\boldsymbol{T}_{C_{t+1}}$
  - Will this Transform be 100% correct or contain errors?

CRICOS No.00213J

Should we use every camera frame or skip some?

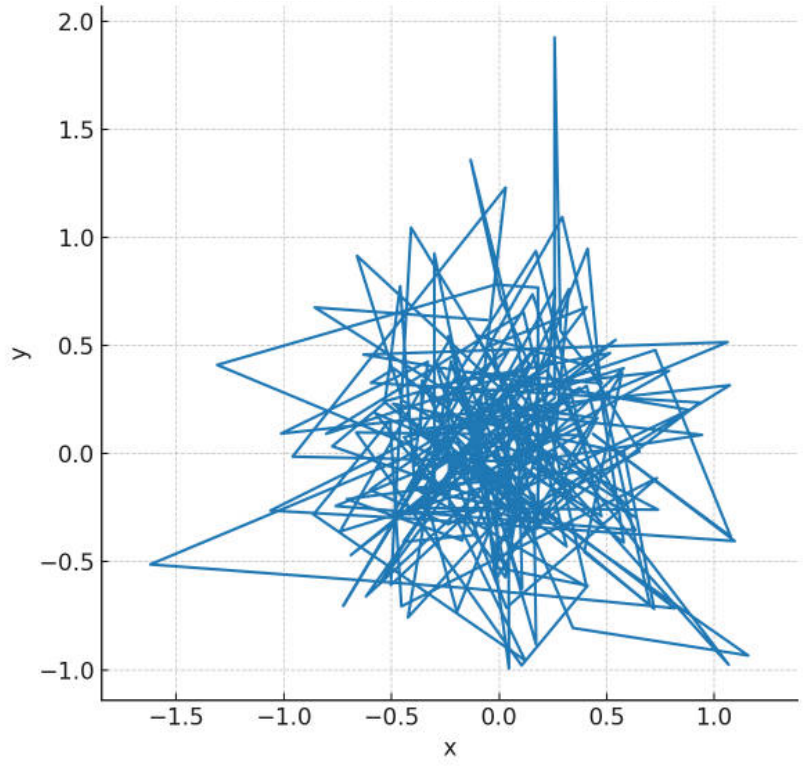
- Imagine your favourite visual odometry method estimates the frame-to-frame motion of the camera: ${}^{C_t}\boldsymbol{T}_{C_{t+1}}$
  - Will this Transform be 100% correct or contain errors?

- Imagine your camera does not actually move at all.
  - If you plot your odometry over time, what does it look like after 60 seconds?

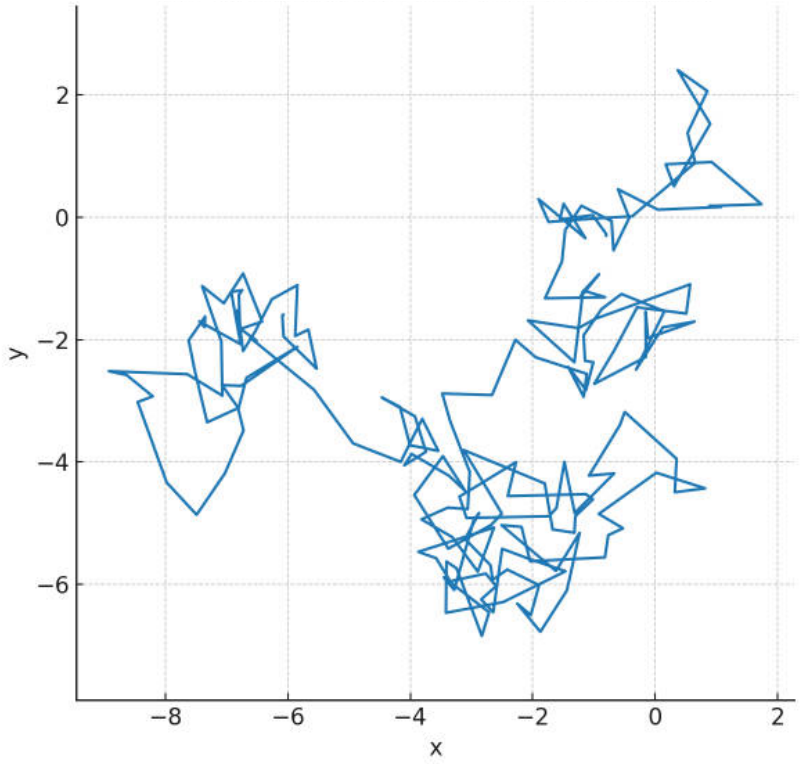
CRICOS No.00213J

Should we use <u>every</u> camera frame or skip some?

- Imagine your camera does not actually move at all.
  - If you plot your odometry over time, what does it look like after 60 seconds?



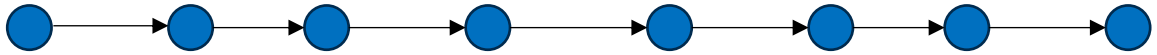
or



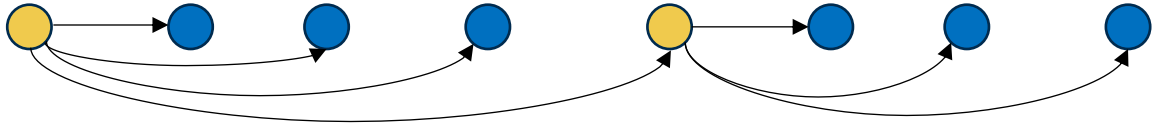
CRICOS No.00213J

Keyframes

- Don’t estimate the motion between every consecutive image pair ${}^{C_{t-1}}\boldsymbol{T}_{C_t}$



- Instead, estimate the motion relative to the last keyframe: ${}^{K_i}\boldsymbol{T}_{C_t}$



When to add a new Keyframe

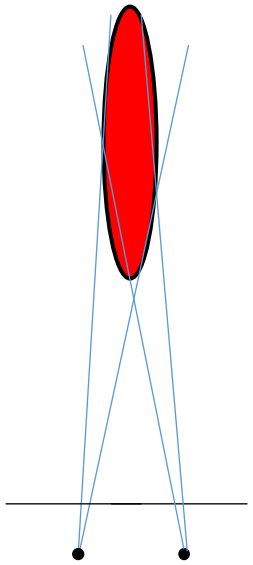
- Heuristics often used:
  - **Camera motion threshold**: new keyframe if the camera moved/rotated enough
  - **Feature tracking quality**: new keyframe if too many features are lost
  - **Time-based**: e.g., every N seconds or frames
- Balance: **Too few keyframes → poor map**, **too many → inefficiency**

CRICOS No.00213J

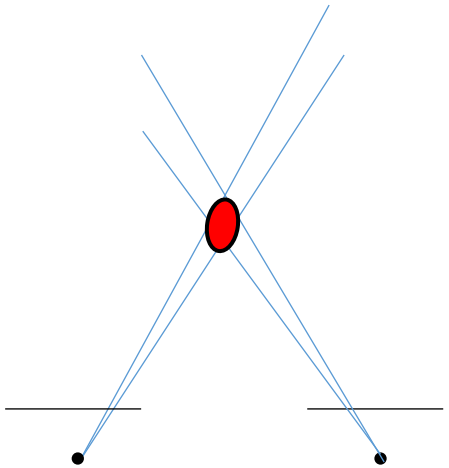
Case Study: Monocular VO (i.e., single camera VO)

2. Keyframe selection (i.e., skipping frames)

- When frames are taken at nearby positions compared to the scene distance, 3D points will exibit large uncertainty



Small baseline → large depth uncertainty



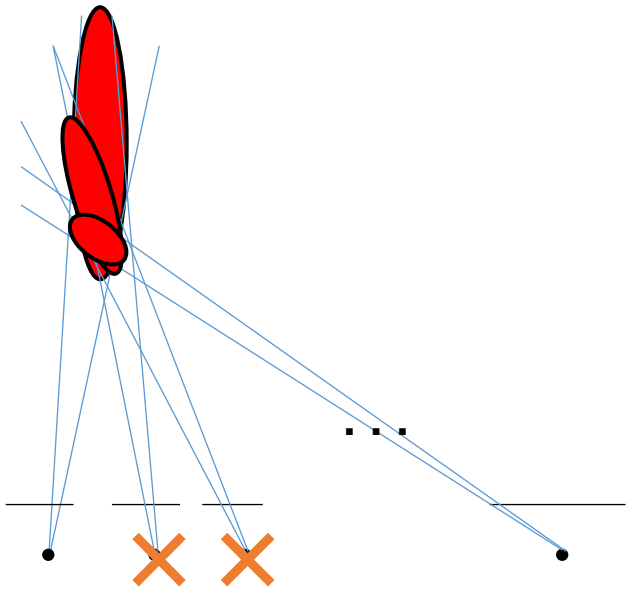
Large baseline → small depth uncertainty

Slide courtesy of Davide Scaramuzza

Case Study: Monocular VO (i.e., single camera VO)

2. Keyframe selection (i.e., skipping frames)

- When frames are taken at nearby positions compared to the scene distance, 3D points will exibit large uncertainty
- One way to avoid this consists of **skipping frames** until the average uncertainty of the 3D points, normalized by the average distance from the scene, falls below a certain threshold. The selected frames are called ***keyframes***
- **Rule of the thumb:** add a keyframe when $\frac{\textit{keyframe distance}}{\textit{average-depth}} > \textit{threshold}$ (usually 10-20 %)



Slide courtesy of Davide Scaramuzza



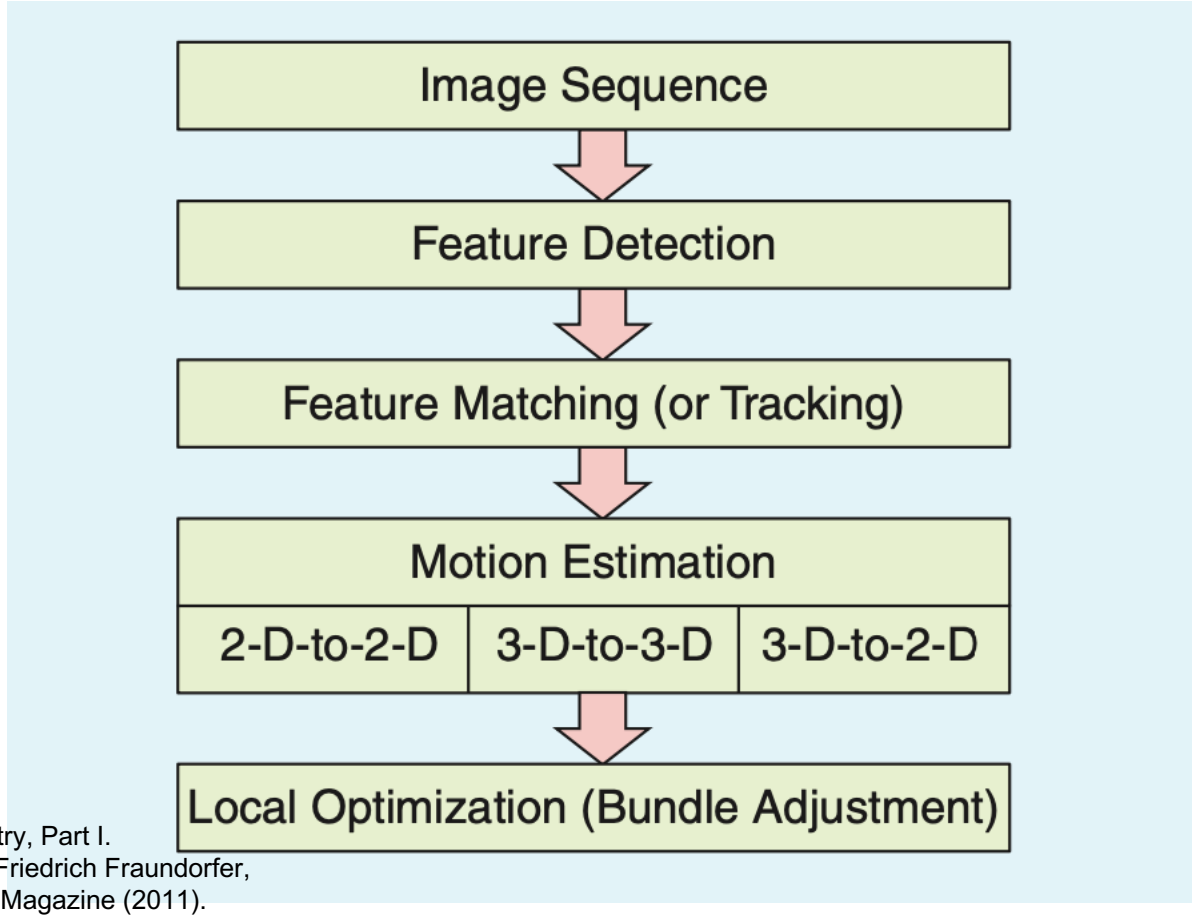
A Case Study

Case Study: Monocular Visual Odometry in the Oculus VR Headset



CRICOS No.00213J

Case Study: Monocular Visual Odometry with PTAM



Tutorial on Visual Odometry, Part I.
Davide Scaramuzza and Friedrich Fraundorfer,
Robotics and Automation Magazine (2011).



Parallel Tracking and Mapping for Small AR Workspaces

Georg Klein* David Murray†

Active Vision Laboratory
Department of Engineering Science
University of Oxford

ABSTRACT

This paper presents a method of estimating camera pose in an unknown scene. While this has previously been attempted by adapting SLAM algorithms developed for robotic exploration, we propose a system specifically designed to track a hand-held camera in a small AR workspace. We propose to split tracking and mapping into two separate tasks, processed in parallel threads on a dual-core computer: one thread deals with the task of robustly tracking erratic hand-held motion, while the other produces a 3D map of point features from previously observed video frames. This allows the use of computationally expensive batch optimisation techniques not usually associated with real-time operation: The result is a system that produces detailed maps with thousands of landmarks which can be tracked at frame-rate, with an accuracy and robustness rivalling that of state-of-the-art model-based systems.

1 INTRODUCTION

The majority of Augmented Reality (AR) systems operate with prior knowledge of the user's environment - i.e, some form of map. This could be a map of a city, CAD model of a component requiring maintenance, or even a sparse map of fiducials known to be present in the scene. The application then allows the user to interact with this environment based on prior information on salient parts of this model (e.g. "This location is of interest" or "remove this nut from this component"). If the map or model provided is comprehensive, registration can be performed directly from it, and this is the common approach to camera-based AR tracking.

Unfortunately, a comprehensive map is often not available, often a small map of only an object of interest is available - for example, a single physical object in a room or a single fiducial marker. Tracking is then limited to the times when this known feature can be measured by some sensor, and this limits range and quality of registration. This has led to the development of a class of techniques known (in the AR context) as *extensible tracking* [21, 14, 4, 28, 2] in which the system attempts to add previously unknown scene elements to its initial map, and these then provide registration even when the original map is out of sensing range. In [4], the initial map is minimal, consisting only of a template which provides metric scale; later versions of this monocular SLAM algorithm can now operate without this initialisation template.

The logical extension of extensible tracking is to track in scenes without any prior map, and this is the focus of this paper. Specifically, we aim to track a calibrated hand-held camera in a previously unknown scene without any known objects or initialisation target, while building a map of this environment. Once a rudimentary map has been built, it is used to insert virtual objects into the scene, and these should be accurately registered to real objects in the environment.

Since we do not use a prior map, the system has no deep understanding of the user's environment and this precludes many task-based AR applications. One approach for providing the user with meaningful augmentations is to employ a *remote expert* [4, 16] who can annotate the generated map. In this paper we take a different approach: we treat the generated map as a sandbox in which virtual simulations can be created. In particular, we estimate a dominant plane (a virtual ground plane) from the mapped points - an example of this is shown in Figure 1 - and allow this to be populated with virtual characters. In essence, we would like to transform any flat (and reasonably textured) surface into a playing field for VR simulations (at this stage, we have developed a simple but fast-paced action game). The hand-held camera then becomes both a viewing device and a user interface component.

To further provide the user with the freedom to interact with the simulation, we require fast, accurate and robust camera tracking, all while refining the map and expanding it if new regions are explored. This is a challenging problem, and to simplify the task somewhat we have imposed some constraints on the scene to be tracked: it should be mostly *static*, i.e. not deformable, and it should be *small*. By small we mean that the user will spend most of his or her time in the same place: for example, at a desk, in one corner of a room, or in front of a single building. We consider this to be compatible with a large number of workspace-related AR applications, where the user is anyway often tethered to a computer. Exploratory tasks such as running around a city are not supported.

The next section outlines the proposed method and contrasts this to previous methods. Subsequent sections describe in detail the method used, present results and evaluate the method's performance.

*e-mail: gk@robots.ox.ac.uk
†e-mail:dwm@robots.ox.ac.uk


Grow
Grow!
Reset
Pause
Teachers
F
G
C
V
B
N
M

PTAM Mapping

6 MAPPING

This section describes the process by which the 3D point map is built. Map-building occurs in two distinct stages: First, an initial map is built using a stereo technique. After this, the map is continually refined and expanded by the mapping thread as new keyframes are added by the tracking systems. The operation of the mapping

PTAM Map Initialisation

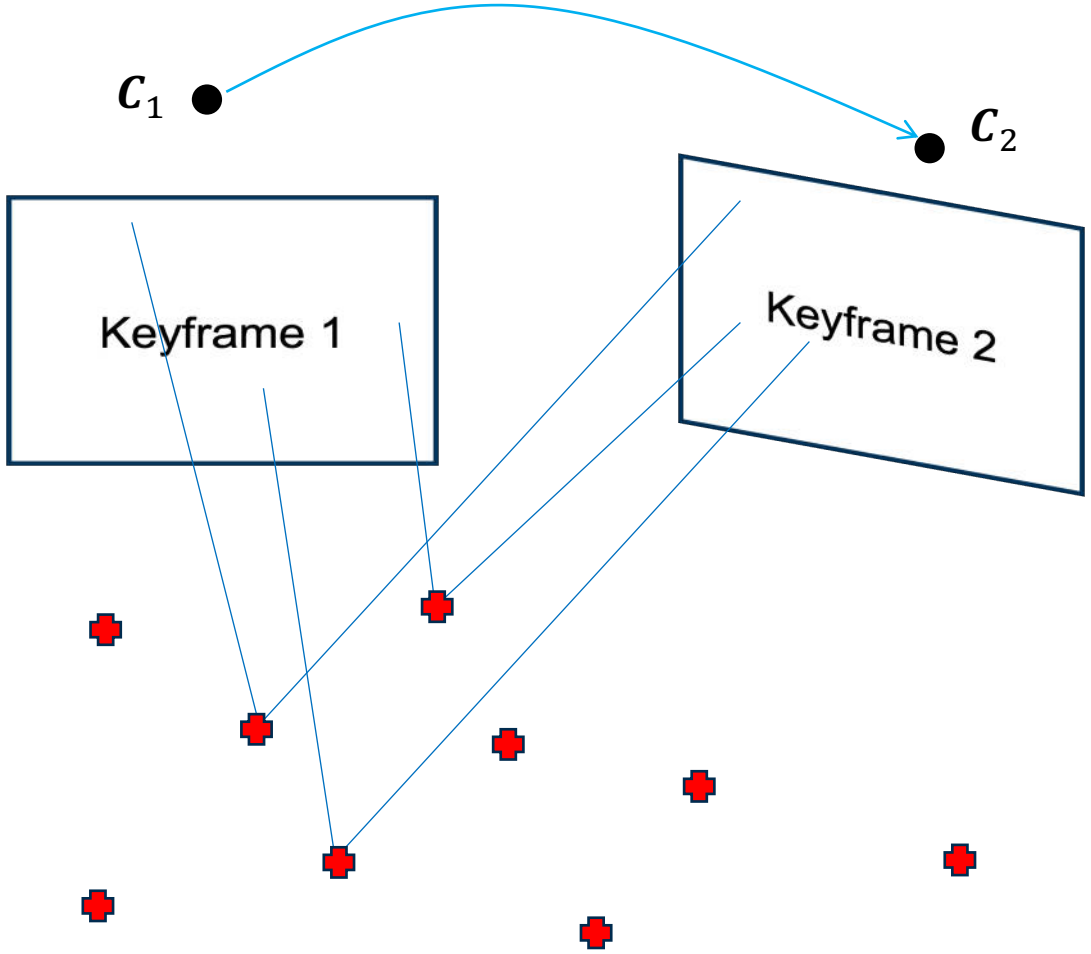
6.1 Map initialisation

When the system is first started, we employ the five-point stereo algorithm of [27] to initialise the map in a manner similar to [20, 18, 9]. User cooperation is required: the user first places the camera above the workspace to be tracked and presses a key. At this stage, they system's first keyframe is stored in the map, and 1000 2D patch-tracks are initialised in the lowest pyramid level at salient image locations (maximal FAST corners.) The user then smoothly translates (and possibly rotates) the camera to a slightly offset position makes a further key-press. The 2D patches are tracked through the smooth motion, and the second key-press thus provides a second keyframe and feature correspondences from which the five-point algorithm and RANSAC can estimate an essential matrix and triangulate the base map. The resulting map is refined through bundle adjustment.

This initial map has an arbitrary scale and is aligned with one camera at the origin. To enable augmentations in a meaningful place and scale, the map is first scaled to metric units. This is done by assuming that the camera translated 10cm between the stereo pair. Next, the map is rotated and translated so that the dominant detected plane lies at z=0 in the world. This is done by RANSAC: many sets of three points are randomly selected to hypothesise a plane while the remaining points are tested for consensus. The winning hypothesis is refined by evaluating the spatial mean and variance of the consensus set, and the smallest eigenvector of the covariance matrix forms the detected plane normal.

Including user interaction, map initialisation takes around three seconds.

PTAM Map Initialisation



- Find 1000 interest points
- Move the camera, track points

- Five-point Essential Matrix estimation with RANSAC
- Triangulate 3D points
- Refinement with Bundle Adjustment
- Scale factor: assume 10cm movement

- Find dominant plane (RANSAC), align map so that dominant plane aligns with XY plane (z=0)

CRICOS No.00213J

PTAM Tracking

5 TRACKING

This section describes the operation of the point-based tracking system, with the assumption that a map of 3D points has already been created. The tracking system receives images from the hand-held video camera and maintains a real-time estimate of the camera pose relative to the built map. Using this estimate, augmented graphics can then be drawn on top of the video frame. At every frame, the system performs the following two-stage tracking procedure:

1. A new frame is acquired from the camera, and a prior pose estimate is generated from a motion model.

2. Map points are projected into the image according to the frame's prior pose estimate.

3. A small number (50) of the coarsest-scale features are searched for in the image.

4. The camera pose is updated from these coarse matches.

5. A larger number (1000) of points is re-projected and searched for in the image.

6. A final pose estimate for the frame is computed from all the matches found.

PTAM Tracking

5.2 Camera pose and projection

To project map points into the image plane, they are first transformed from the world coordinate frame to the camera-centred coordinate frame $\mathcal{C}$. This is done by left-multiplication with a 4×4 matrix denoted $E_{\mathcal{CW}}$, which represents camera pose.

$$\boldsymbol{p}_{j\mathcal{C}} = E_{\mathcal{CW}}\boldsymbol{p}_{j\mathcal{W}} \tag{1}$$

The subscript $\mathcal{CW}$ may be read as "frame $\mathcal{C}$ from frame $\mathcal{W}$". The matrix $E_{\mathcal{CW}}$ contains a rotation and a translation component and is a member of the Lie group SE(3), the set of 3D rigid-body transformations.

To project points in the camera frame into image, a calibrated camera projection model CamProj() is used:

$$\begin{pmatrix} u_i \\ v_i \end{pmatrix} = \mathrm{CamProj}(E_{\mathcal{CW}}\boldsymbol{p}_{i\mathcal{W}}) \tag{2}$$

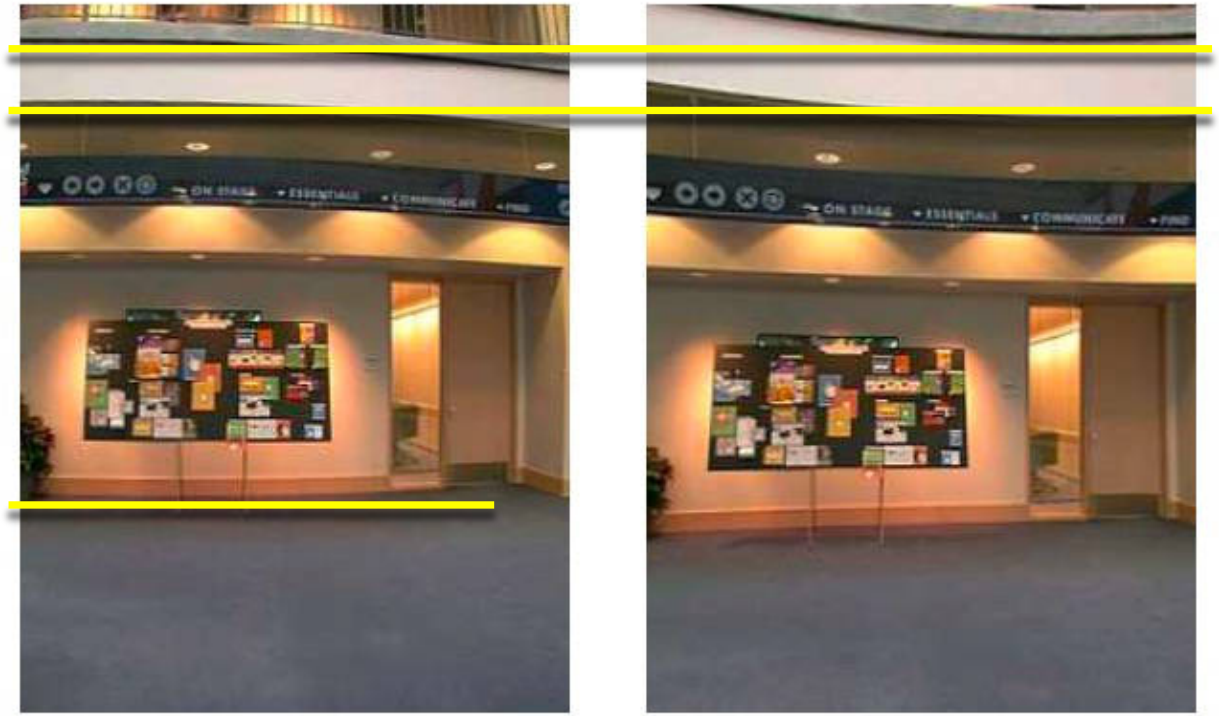
We employ a pin-hole camera projection function which supports lenses exhibiting barrel radial distortion. The radial distortion model which transforms $r \rightarrow r'$ is the FOV-model of [6]. The camera parameters for focal length (f_u, f_v), principal point (u_0, v_0) and distortion (ω) are assumed to be known:

$$\mathrm{CamProj}\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} u_0 \\ v_0 \end{pmatrix} + \begin{bmatrix} f_u & 0 \\ 0 & f_v \end{bmatrix} \frac{\mathrm{r}'}{\mathrm{r}} \begin{pmatrix} \frac{x}{z} \\ \frac{y}{z} \end{pmatrix} \tag{3}$$

$$r = \sqrt{\frac{x^2 + y^2}{z^2}} \tag{4}$$

$$r' = \frac{1}{\omega}\arctan(2r\tan\frac{\omega}{2}) \tag{5}$$

Remember: Lens Distortion





Radial lens distortions: (left) barrel, (middle) pincushion, and (right) fisheye.

Szeliski, Computer Vision: Algorithms and Applications (2nd Edition), Springer Verlag

PTAM Tracking

5.4 Pose update

Given a set S of successful patch observations, a camera pose update can be computed. Each observation yields a found patch position $(\hat{u}\ \hat{v})^T$ (referred to level zero pixel units) and is assumed to have measurement noise of $\sigma^2 = 2^{2l}$ times the 2×2 identity matrix (again in level zero pixel units). The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

$$\boldsymbol{\mu}' = \underset{\boldsymbol{\mu}}{\operatorname{argmin}} \sum_{j \in S} \operatorname{Obj}\left(\frac{|\boldsymbol{e}_j|}{\sigma_j}, \sigma_T\right) \tag{8}$$

where $\boldsymbol{e}_j$ is the reprojection error vector:

$$\boldsymbol{e}_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \operatorname{CamProj}\left(\exp\left(\boldsymbol{\mu}\right) E_{\mathcal{CW}} \boldsymbol{p}_j\right). \tag{9}$$

$\operatorname{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution's standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

PTAM Tracking

A fundamental requirement of the tracking (and also the mapping) system is the ability to differentiate Eq. 2 with respect to changes in camera pose $E_{\mathcal{CW}}$. Changes to camera pose are represented by left-multiplication with a 4×4 camera motion M:

$$E'_{\mathcal{CW}} = ME_{\mathcal{CW}} = \exp(\boldsymbol{\mu})E_{\mathcal{CW}} \tag{6}$$

where the camera motion is also a member of SE(3) and can be minimally parametrised with a six-vector $\boldsymbol{\mu}$ using the exponential map. Typically the first three elements of $\boldsymbol{\mu}$ represent a translation and the latter three elements represent a rotation axis and magnitude. This representation of camera state and motion allows for trivial differentiation of Eq. 6, and from this, partial differentials of Eq. 2 of the form $\frac{\partial u}{\partial \mu_i}, \frac{\partial v}{\partial \mu_i}$ are readily obtained in closed form. Details of the Lie group SE(3) and its representation may be found in [29].

5.4 Pose update

Given a set S of successful patch observations, a camera pose update can be computed. Each observation yields a found patch position $(\hat{u}\ \hat{v})^T$ (referred to level zero pixel units) and is assumed to have measurement noise of $\sigma^2 = 2^{2l}$ times the 2×2 identity matrix (again in level zero pixel units). The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

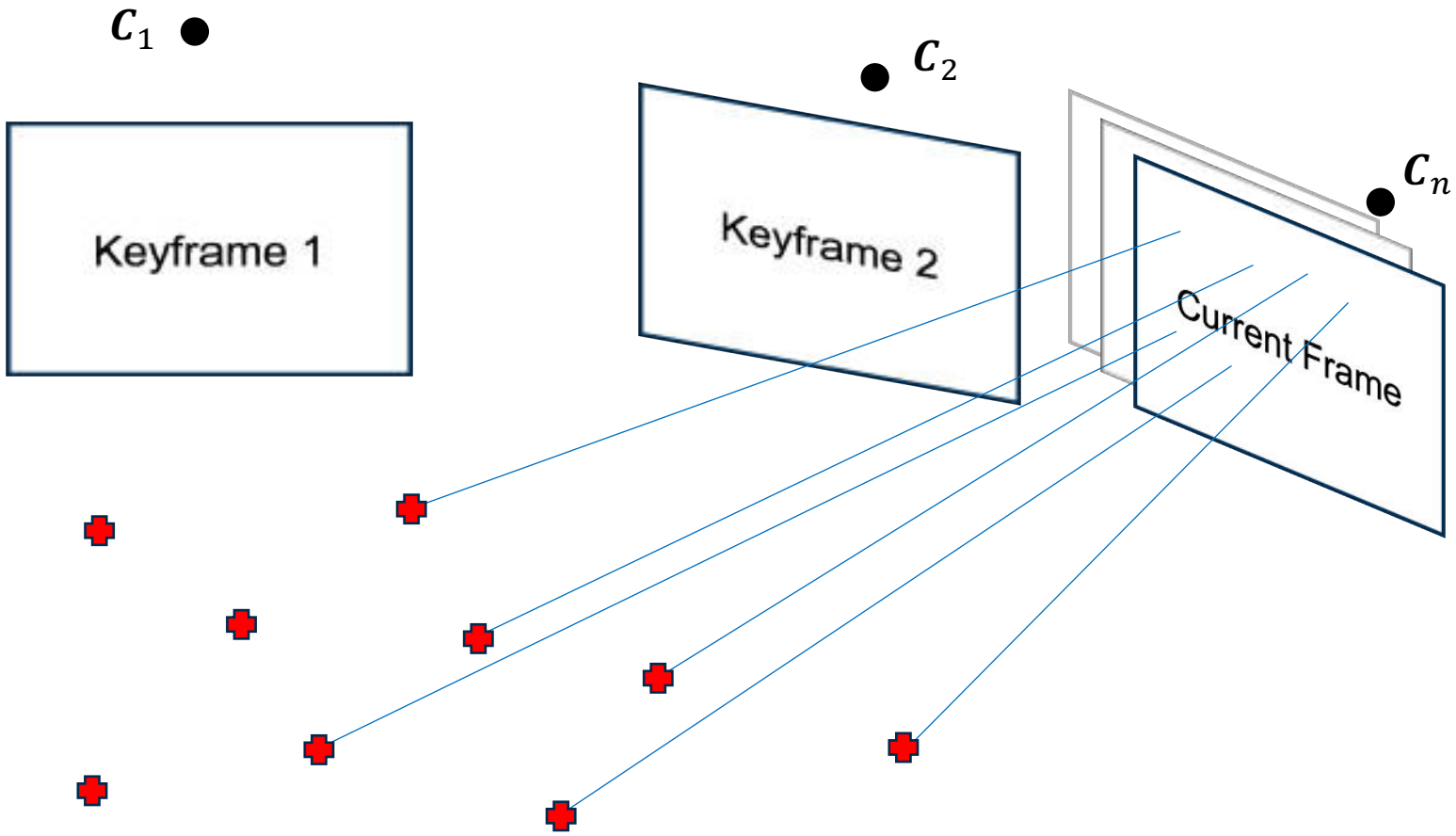
$$\boldsymbol{\mu}' = \underset{\boldsymbol{\mu}}{\operatorname{argmin}} \sum_{j \in S} \operatorname{Obj}\left(\frac{|\boldsymbol{e}_j|}{\sigma_j}, \sigma_T\right) \tag{8}$$

where $\boldsymbol{e}_j$ is the reprojection error vector:

$$\boldsymbol{e}_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \operatorname{CamProj}\left(\exp\left(\boldsymbol{\mu}\right) E_{\mathcal{CW}} \boldsymbol{p}_j\right). \tag{9}$$

$\operatorname{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution's standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

PTAM Tracking

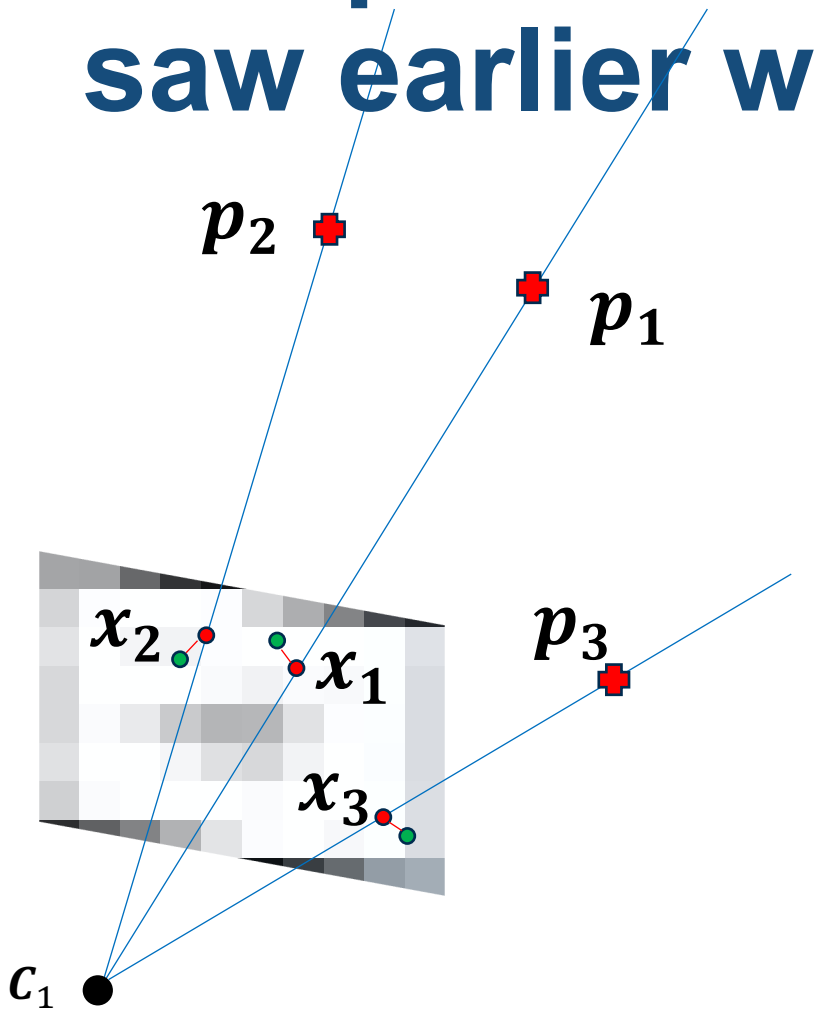


Tracking:

Update current pose based on 3D-2D correspondences.

CRICOS No.00213J

Compare the “generic” loss function we saw earlier with PTAM’s approach.



$$\text{argmin}_{R,t} \sum_i \left\| x_\tau^i - K(R \cdot p^i{}_{\tau-1} + t) \right\|^2$$

vs

The pose update is computed iteratively by minimising a robust objective function of the reprojection error:

$$\mu' = \underset{\mu}{\text{argmin}} \sum_{j \in S} \text{Obj}\left(\frac{|e_j|}{\sigma_j}, \sigma_T\right) \tag{8}$$

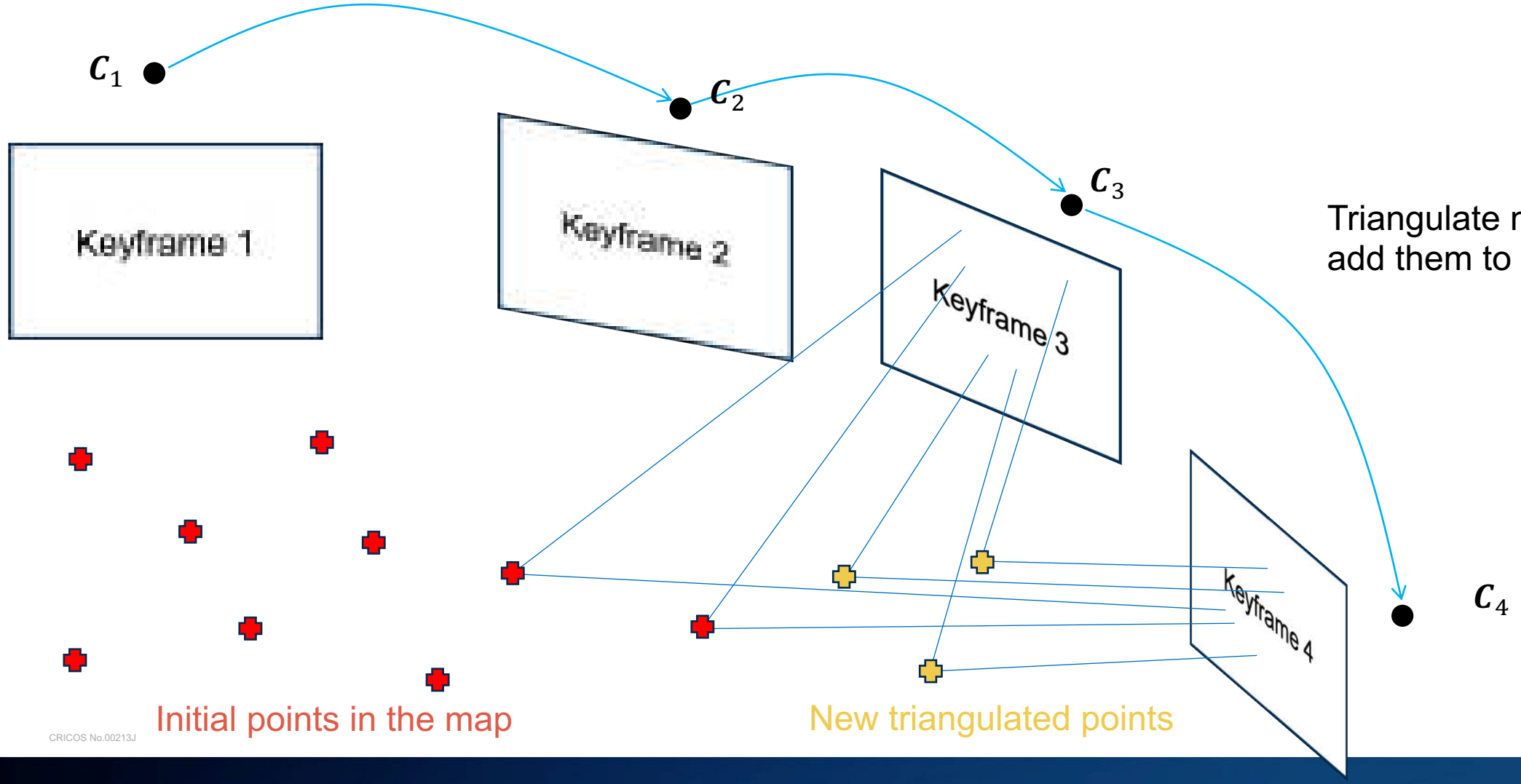
where e_j is the reprojection error vector:

$$e_j = \begin{pmatrix} \hat{u}_j \\ \hat{v}_j \end{pmatrix} - \text{CamProj}\left(\exp(\mu)\, E_{\mathcal{CW}} p_j\right). \tag{9}$$

$\text{Obj}(\cdot, \sigma_T)$ is the Tukey biweight objective function [13] and σ_T a robust (median-based) estimate of the distribution’s standard deviation derived from all the residuals. We use ten iterations of reweighted least squares to allow the M-estimator to converge from any one set of measurements.

CRICOS No.00213J

PTAM Map Update



Triangulate new points and add them to the map.

Initial points in the map

New triangulated points

CRICOS No.00213J



Outlook

Outlook

- Week 6:
  - Pose Graphs and Factor Graphs
  - How to actually solve the optimization problems we saw today
- Week 7:
  - Visual SLAM
  - Visual Inertial Odometry
  - Evaluation of VO and SLAM
- Week 8 onwards:
  - Deep learning for Robotic Vision

CRICOS No.00213J